ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

NEUMANN JÁNOS INFORMATIKAI KAR

DIPLOMAMUNKA

OE-NIK Hallgató neve: **Gipsz Jakab**

20.. Hallgató törzskönyvi száma: **T-000123/FI12904**

Óbudai Egyetem
Neumann János Informatikai Kar
…….. Intézet

SZAKDOLGOZAT vagy DIPLOMAMUNKA
FELADATLAP

Hallgató neve: **Gipsz Jakab**
Törzskönyvi száma: NIK-O-NI-02-33

A dolgozat címe:

A gammasugarak hatása a százszorszépekre

The influence of gamma rays to the moonflowers

Intézményi konzulens: ……
Külső konzulens: ……

Beadási határidő: ……

A záróvizsga tárgyai: ……
……

A feladat:

A dolgozatnak tartalmaznia kell:

1. **…**
2. **…**
3. **…**
4. **…**
5. **…**

Ph

.....
intézetigazgató

A dolgozat (OE TVSz. 55.§ 9 és 9a. pont szerinti) elévülésének határideje: 20xx.

A dolgozatot beadásra alkalmasnak tartom:

.....	.....
külső konzulens	intézményi konzulens

HALLGATÓI NYILATKOZAT

Alulírott hallgató kijelentem, hogy a szakdolgozat / diplomamunka saját munkám eredménye, a felhasznált szakirodalmat és eszközöket azonosíthatóan közöltem. Az elkészült szakdolgozatomban / diplomamunkámban található eredményeket az egyetem és a feladatot kiíró intézmény saját céljára térítés nélkül felhasználhatja.

Budapest, ÉÉÉÉ. hónap nap.

.....

hallgató aláírása

ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

Neumann János Informatikai Kar

KONZULTÁCIÓS NAPLÓ

Hallgató neve:	Neptun Kód:	Tagozat:
GIPSZ JAKAB	OENIK1	NAPPALI

Telefon: +36

Levelezési cím (pl.: lakcím): itt szerepel a levelezési cím, irányítószámmal együtt

Szakdolgozat / Diplomamunka[1] címe magyarul:

Itt szerepel a diplomamunka címe magyarul

Szakdolgozat / Diplomamunka[2] címe angolul:

Itt szerepel a diplomamunka címe angolul

Intézményi konzulens: .....

Külső konzulens: .....

Kérjük, hogy az adatokat nyomtatott nagy betűkkel írja!

Alk.	Dátum	Tartalom	Aláírás
1.			
2.			
3.			
4.			

A Konzultációs naplót összesen 4 alkalommal, az egyes konzultációk alkalmával kell láttamoztatni bármelyik konzulenssel.

A hallgató a Szakdolgozat / Szakdolgozat I. / Szakdolgozat II. / Projektlabor 1 / Projektlabor 2 / Projektlabor 3 / Záródolgozati projekt / Diplomamunka I / Diplomamunka II / Diplomamunka III / Diplomamunka IV [3] tantárgy követelményét teljesítette, beszámolóra / védésre [4]bocsátható.

A konzulens által javasolt érdemjegy: érdemjegy (n)

Budapest, 202X. hónap nap

.....

Intézményi konzulens

[1] Megfelelő aláhúzandó!
[2] Megfelelő aláhúzandó!
[3] Megfelelő aláhúzandó!
[4] Megfelelő aláhúzandó!

ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

Neumann János Informatikai Kar

KONZULTÁCIÓS NAPLÓ

Hallgató neve:	Neptun Kód:	Tagozat:
GIPSZ JAKAB	OENIK1	NAPPALI

Telefon:	Levelezési cím (pl.: lakcím):
+36	itt szerepel a levelezési cím, irányítószámmal együtt

Szakdolgozat / Diplomamunka[1] címe magyarul:

Itt szerepel a diplomamunka címe magyarul

Szakdolgozat / Diplomamunka[2] címe angolul:

Itt szerepel a diplomamunka címe angolul

Intézményi konzulens: ..... Külső konzulens: .....

Kérjük, hogy az adatokat nyomtatott nagy betűkkel írja!

Alk.	Dátum	Tartalom	Aláírás
1.			
2.			
3.			
4.			

A Konzultációs naplót összesen 4 alkalommal, az egyes konzultációk alkalmával kell láttamoztatni bármelyik konzulenssel.

A hallgató a Szakdolgozat / Szakdolgozat I. / Szakdolgozat II. / Projektlabor 1 / Projektlabor 2 / Projektlabor 3 / Záródolgozati projekt / Diplomamunka I / Diplomamunka II / Diplomamunka III / Diplomamunka IV [3] tantárgy követelményét teljesítette, beszámolóra / védésre [4]bocsátható.

A konzulens által javasolt érdemjegy: érdemjegy (n)

Budapest, 202X. hónap nap

.....

Intézményi konzulens

[1] Megfelelő aláhúzandó!
[2] Megfelelő aláhúzandó!
[3] Megfelelő aláhúzandó!
[4] Megfelelő aláhúzandó!

ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

Neumann János Informatikai Kar

KONZULTÁCIÓS NAPLÓ

Hallgató neve:	Neptun Kód:	Tagozat:
GIPSZ JAKAB	OENIK1	NAPPALI

Telefon: +36

Levelezési cím (pl.: lakcím): itt szerepel a levelezési cím, irányítószámmal együtt

Szakdolgozat / Diplomamunka[1] címe magyarul:

Itt szerepel a diplomamunka címe magyarul

Szakdolgozat / Diplomamunka[2] címe angolul:

Itt szerepel a diplomamunka címe angolul

Intézményi konzulens: .....

Külső konzulens: .....

Kérjük, hogy az adatokat nyomtatott nagy betűkkel írja!

Alk.	Dátum	Tartalom	Aláírás
1.			
2.			
3.			
4.			

A Konzultációs naplót összesen 4 alkalommal, az egyes konzultációk alkalmával kell láttamoztatni bármelyik konzulenssel.

A hallgató a Szakdolgozat / Szakdolgozat I. / Szakdolgozat II. / Projektlabor 1 / Projektlabor 2 / Projektlabor 3 / Záródolgozati projekt / Diplomamunka I / Diplomamunka II / Diplomamunka III / Diplomamunka IV [3] tantárgy követelményét teljesítette, beszámolóra / védésre [4]bocsátható.

A konzulens által javasolt érdemjegy: érdemjegy (n)

Budapest, 202X. hónap nap

.....

Intézményi konzulens

[1] Megfelelő aláhúzandó!
[2] Megfelelő aláhúzandó!
[3] Megfelelő aláhúzandó!
[4] Megfelelő aláhúzandó!

ABSZTRAKT

Absztrakt magyarul.

ABSTRACT

Abstract in english.

TARTALOMJEGYZÉK

1 BEVEZETÉS 3

2 HASONLÓ FEJLESZTÉSEK VIZSGÁLATA 4

2.1 Klasszikus képfeldolgozáson alapuló úthibadetektálás 4

2.1.1 Pothole detection in asphalt pavement images 4

2.1.2 Pothole Detection with Image Processing and Spectral Clustering . . . 6

2.1.3 Pothole detection on asphalt pavements from 2D-colour pothole images using fuzzy c-means clustering and morphological reconstruction 8

2.1.4 Watershed-alapú valós idejű képfeldolgozás aszfaltúton található többszörös kátyúk észlelésére 10

2.2 Konvolúciós neurális hálózatokon alapuló úthibadetektálás 11

2.2.1 Kétfázisú objektumdetektálás Faster R-CNN 12

2.2.2 Egyfázisú objektumdetektálás You Only Look Once v1 14

2.2.3 YOLO metrikák 16

2.2.4 YOLOv5, YOLOv8 és YOLOv10 fejlődése 17

2.2.5 Rendelkezésre álló nyílt adathalmazok és versenyek a témában 19

2.2.6 Transfer Learning és fine-tuning a gyakorlatban 21

2.3 Detektált hibák lokalizálása 22

2.3.1 Geometriai képalkotás elméleti háttere 23

2.3.2 Geometriai transzformációk 25

2.4 Nyílt forráskódú szimulációs környezetek. 26

2.4.1 CARLA 27

2.4.2 BeamNG.tech. 27

2.5 Szintetikus kátyú és teszt pálya generálás 27

2.5.1 3D modellezés 27

2.5.2 Decals (matricák) 28

3 SPECIFIKÁCIÓ 30
3.1 Program funkciói 30
3.1.1 Program célja 30
3.1.2 Elérhető funkciók részletesen 30
3.2 Követelmények 31
3.2.1 Követelmények kifejtve 31
3.2.2 Követelmények felsorolás jelleggel 33
3.3 Megvalósítási módszertan 34
3.3.1 Környezetérzékelés: Iteratív Kísérleti Fejlesztés 34
3.3.2 Lokalizáció: Analitikus Modellalapú Rekonstrukció 34
4 RÉSZLETES RENDSZERTERV 35
4.1 Rendszer architektúra. 35
4.1.1 Adatfolyam 35
4.2 Fejlesztői környezet 35
4.2.1 Fejlesztői munkaállomás 35
4.2.2 Szimulációs környezet 36
4.2.3 Fejlesztői környezet, programozási nyelv, könyvtárak és eszközök . . . 36
4.3 Modulok felépítése. 37
4.3.1 Adatmodell és struktúrák 37
4.3.2 Fő vezérlési ciklus 38
4.3.3 Előfeldolgozás 39
4.3.4 Kátyú lokalizálás 40
4.3.5 Megjelenítés 42
5 IRODALOMJEGYZÉK 43
6 ÁBRAJEGYZÉK. 48
7 TÁBLAJEGYZÉK 49

1 BEVEZETÉS

Az autóutak burkolatának megléte és minősége rendkívül fontos a mai világban, ahol nap mint nap rengeteg jármű használja ezeket az utakat közlekedésre. A használatból és környezeti tényezőktől (esővíz beszivárgás, fagyás, olvadás) a megépítés és vagy karbantartás nem megfelelő minőségéből fakadóan úthibák képződhetnek és az idő múlásával ezek egyre súlyosodhatnak, ha nincsenek időben felmérve és megfelelően javítva [1] .Ezt a jelenséget nevezzük kátyúnak, ami az útburkolat felszínen található durva, egyenletlen tál alakú mélyedéshez hasonlítható, amelynek szélessége és mélysége is széles skálán mozog [2], [3].

A kátyúk a közúti infrastruktúra olyan hibái, amelyek világszerte jelentős, problémákat okoznak. Talán a leginkább szembetűnő probléma az utazási menetkomfort romlása amikor a jármű belehajt egy kátyúba. Továbbá közlekedésbiztonsági veszélyt hordoznak magukban akár a jármű feletti tejes uralom elvesztéséhez vezethetnek, különösen nagy sebességnél, főként a kiszolgáltatott járművek (például kerékpárosok és motorosok) körében ezzel növelve a balesetek bekövetkezési kockázatát [4]. Gazdasági szempontból a kátyúk jelentős pénzügyi terhet jelentenek az úthálózat fenntartó szervezeteknek és az utat használó közlekedők számára egyaránt. Míg a közlekedő járművek esetén jelentősen befolyásolják és növelhetik a járművek karbantartási költségeit azzal, hogy csökkentik az abroncsok, futómű alkatrészek és egyéb mechanikus alkatrész élettartamát. Addig a kátyúk detektálása és javítása hatalmas összegeket emészthet fel kormányzati oldalról nézve. Az Egyesült Királyság kormánya olyan közleményt adott ki, amelyben azt írták, hogy 2020 és 2025 között időszakban több mint 5 milliárd fontot terveznek kátyú detektálásra és javításra fordítani [3]. „Ezenkívül a Világbank szerint az optimális állapotban lévő utak mennyisége egy ország gazdasági erejének és versenyképességének mutatójaként használható.” [5]

Az előbb felsoroltak miatt a kátyúkkal kapcsolatos kutatások a mai napig aktívak, és engem is arra sarkallott, hogy érdemes ezzel a témával foglalkozni. Illetve van még egy tényező, amikor autót vezetek gyakran találom magam szembe kátyúkkal az utakon ezeket a lehetőségekhez mérten rendre próbálok kikerülni úgy, hogy ezzel másokat ne veszélyeztessek az úton, így próbálom óvni az autó szerkezeti állapotát és a saját magam vagy adott esetben az utasaim menetkomfortját. Bármennyire is igyekszem figyelni vezetés közben az utat nyilvánvalóan előfordul, hogy későn, vagy egyáltalán észre se veszem a kátyút az úton és akarva akaratlanul áthajtok rajta.

Meglátásom szerint kátyúk mindig is lesznek amíg javításuk ennyire drága és körülményes. Ebbe beletörődve és autóval az úton töltött tapasztalataimat figyelembe véve jutottam arra, hogy valós időben szeretném a kátyúkat megbízhatóan felismerni, majd azok minél pontosabb elhelyezkedését megállapítani az úton belül.

2 HASONLÓ FEJLESZTÉSEK VIZSGÁLATA

A projektem két könnyen körül határolható problémát fog megoldani, így magát az irodalom kutatásit is ily módon fogom megközelíteni. Az elkövetkezendőkben először az útfelületi hibák detektálásával, azt követően a detektált hibák úton belüli lokalizálásával fogok foglalkozni.

2.1 Klasszikus képfeldolgozáson alapuló úthibadetektálás

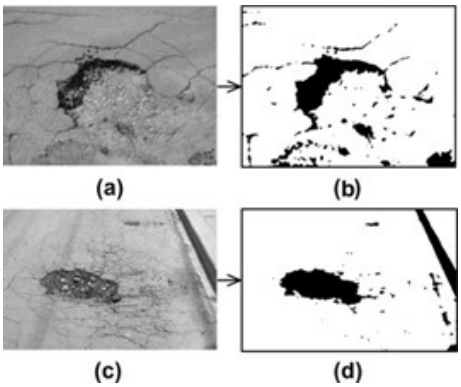
2.1.1 Pothole detection in asphalt pavement images

Koch Christian és Brilakis Ioannis egy 2D-s képeken alapuló eljárást javasoltak 2011-ben, amely a kátyúk három fő vizuális jellemzőjére épít. Az első jellemző az árnyék, ezáltal a kátyúk általában sötétebbek a környezetüknél, alakzatuk többnyire ellipszishez hasonló, és a textúrájuk durvább, szemcsésebb, mint a sima aszfalt.

Szegmentálás lépésesei a következőek, a bemeneti RGB képet szürkeárnyalatossá alakítják. Ezt követően 5×5-ös medián szűrőt csúsztatnak végig a szürkeárnyalatos képen, ezzel az apró pixelhibákat, zajt távolítja el anélkül, hogy az éleket túlságosan elmosná. A zajszűrés után a küszöbölés következik, itt ahelyett, hogy standard módszert (például: Otsu) használnának, ők egy „triangle algorithm” (háromszög alapú algoritmust) futtatnak a hisztogramon. A legsötétebb, azaz 0 intenzitású pontból(P_0) húznak egy I egyenes a hisztogram legmagasabb csúcsához (P_{max}), azaz leggyakoribb előfordulású intenzitáshoz. Majd az algoritmus megkeresi azt a pontot a hisztogramon, ami P_0 és P_{max} pontok között van, és a legtávolabb helyezkedik el ettől az I egyenestől. Ennek a pontnak a szürkeárnyalatos értéke lesz a küszöb érték. A mögöttes elgondolás az, hogy a háttér nagyon domináns, tehát a legmagasabb és egyben a legvilágosabb hegycsúcs a hátteret jelöli, míg a kisebb sötétebb dombok, tartományából (például: kátyú, repedés, árnyék) kevesebb van a képen. Az algoritmus azokat az átmeneti tartományokat, völgyeket vizsgálja, amik átmenet jelentenek, a háttér és az előtér között. A küszöbérték meghatározása után, létrehoznak egy bináris képet elválasztva a hátteret az előtértől.

A bináris képen további szűrőket alkalmaznak, a cél az, hogy azokat az alakzatokat eltávolítsák a képről, amik valószínűsíthetően nem kátyúk. A tisztítás lépései a következőek, ha egy fekete (előtér) régión belül, fehér (hátteret) apró „szigeteket” vannak, azokat kitöltik feketével. Azokat a kis fekete foltokat, amik túl kicsik (rövid repedések, kisebb zajok a háttéren) törlik a képről. A lineáris alakzatok eltávolítására excentricitás módszer használnak, az excentricitás egy szám 0 és 1 között, ami leírja, hogy az adott alakzat mennyire nyújtott (például: kör excentricitás 0, egyenes vonal excentricitás 1). Tehát az algoritmus végig iterál a fekete alakzatokon és ami egyenes vonalhoz hasonlít azt tör-

li. Még azokat a fekete foltokat távolítják el a képről, amik hozzá érnek a kép széléhez, ugyanis ezek nem teljes alakzatok, így megtéveszthetőek lehetnek a későbbiekben. A szűrés lépések után, már csak olyan fekete szegmensek maradnak a képen, amelyek méretük és alakjuk alapján kátyút reprezentálhatnak. Előfordulhat, hogy az árnyék alakja és nagysága eltérő a kátyún belül, ezt szemlélteti a 2.1. ábra. [6]



2.1. ábra: Kátyún belüli árnyékok szemléltetése [1]

A kátyún belüli árnyék mértékét és kontrasztját leginkább a fényforrás iránya, azaz beesési szöge határozza meg, illetve a kamera perspektívája, ezek a jelenségek jól megfigyelhetőek az 2.1. ábra b és d részén. Jól látható, hogy a d képen a fekete folt maga a teljes kátyú, míg a b képről ez nem mondható el. A b képen csak a kátyú egy bizonyos része árnyékos így nem fedi le a kátyú teljes területét. Ebből kifolyólag a fekete foltokon további geometriai vizsgálatokat végeznek a bináris képeken, hogy megállapítsák azt, hogy melyik esettel állnak szembe (b vagy d) az adott képnél.

A vizsgálat lépései kiszámolják a folt súlypontját majd megnézik, hogy a súlypont a folton belül van e, ezzel megállapítják, hogy az alakzat tömör vagy kifli alakú. Abban az esetben, ha tömör, de nyújtott vagy kifli alakú, akkor megnézik, hogy mennyire hosszúkás és mennyire vékony a vonal. Ehhez excentricitását, leghosszabb átmérőt és területet számolnak. Majd a következőként dönt, ha a súlypont kívül van vagy a forma hosszúkás és vékony akkor az árnyék nem fedi le a kátyú teljes területét. Ellenkező esetben, ha a súlypont belül van, és a forma nem extrém módon hosszúkás, és se nem vékony, akkor az sötét folt a teljes kátyút jelöli.

Következik a textúra elemzése, itt igyekeznek megkülönböztetni a valódi kátyúkat a többi sötét és nagyjából ellipszis alakú foltoktól (például: olajfolt, sima árnyék, egyéb objektum). Ennek meghatározása érdekében a foltok belsejét és a külsejét vizsgálják. Ezt úgy érik el, hogy veszik a bináris képet, majd kiterjesztik morfológiai dilatáció segítségével. Ez azért kell, hogy a lehetséges kátyú közvetlen szélén lévő, bizonytalan pixelek ne számítsanak bele a külső régióba. Az eredeti bináris képből kivonják ezt a kiterjesztett képet, ezentúl kivonják belőle, magának a vizsgált foltnak a területét is. Ami megmarad

az lesz a megbízható, referenciaként felhasználható, ép út textúrája.

Az ötlet az, hogy a kátyú belseje általában érdesebb, mint a sima út. Megnézik az összes pixel szürkeárnyalatos intenzitás értékét az adott folt belsejiben és kiszámolják ezeknek az értékeknek a szórását. A folt területe sima, ha az intenzitások szórása alacsony, érdes, ha a szórás magas. Mivel a kátyú szemcsés ezért spot filter-eket (folt szűrő) használnak az eredeti szürkeárnyalatos képen. Ezek a szűrők az ilyen apró, pontszerű vagy foltszerű mintázatokat találják meg a képen belül.

Összehasonlítás, mindkét folt régióhoz (belső és külső) létrehoznak egy egy 5 dimenziós jellemzővektort. Ennek az 5 eleme a következő, szürkeárnyalati szórás, és a cikkben használt 4db spot filter válaszának szórása. Kiszámolják mindkét vektor hosszát. A vektor hossza itt azt méri, hogy összességében mennyire erős a textúra az adott régióban az 5 mért jellemző alapján. Ha az adott folt belső vektorának a hossza nagyobb, mint a külsőé akkor a belső textúra kátyút jelöl. Ellenkező esetben nem tekinthető kátyúnak az adott folt. [6]

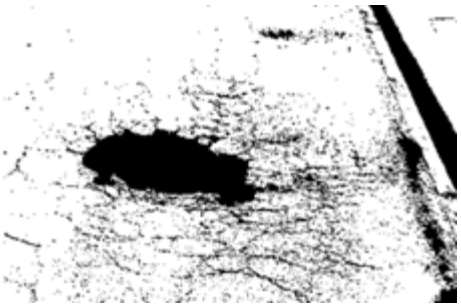
A cikkben leírt módszer fő előnye, hogy textúra összehasonlítást alkalmaz a hamis pozitívumok (pl. olajfoltok, árnyékok) hatékony kiszűrésére, amivel figyelemre méltó, „86%-os pontosságot ér el” klasszikus képfeldolgozó módszerként [6]. A cikk nem közöl konkrét futási időt, de a többlépcsős képfeldolgozási folyamatok miatt, szinte biztos, hogy magas számítási idővel járnak, így egy az egyben nem használható valós idejű kátyú detektálásra. A szegmentálás eredményessége erősen függ a fényviszonyoktól. A szerzők maguk is elismerik, hogy olyan körülmények között, ahol minimális az árnyék (pl. a nap magasan, merőlegesen süt), a módszer hibázhat [6]. A módszer nem adaptív, így valós idejű rendszer esetén nem alkalmazható ugyanis több kritikus paramétert is használnak, amelyeket „manuálisan kellett beállítani egy 50 képből álló tanító adathalmazon” [6].

2.1.2 Pothole Detection with Image Processing and Spectral Clustering

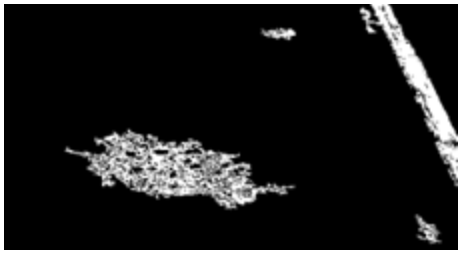
Emir Buza, Alvin Huseinovic és Samir Omanovic egy spektrális klaszterezéses megoldást javasoltak 2013-ban.

Első lépésben klasszikus hisztogram alapú küszöbölést végeznek a képen, az RGB képet szürkeárnyalatos képpé alakítják, majd a hisztogramon Otsu féle küszöbölés alkalmaznak. A végeredmény egy bináris kép lesz, mégpedig úgy, ha az adott pixel kisebb vagy egyenlő a küszöb értékkel(T) akkor objektum, azaz 1, ha az adott pixel nagyobb, mint T akkor háttér, azaz 0. Látható, hogy a 2.2. ábra zajos és a kátyún túl a repedések, árnyékok és egyéb vonalak is feketék.[7]

Ezért a cikk szerzői sajátos megoldást dolgoztak ki, aminek az eredménye a másik 2.3. ábrán látható. Bevezettek egy deltaváltozót, amit úgy számítottak ki, hogy az Otsu kü-



2.2. ábra: Zajos bináris kép [2]



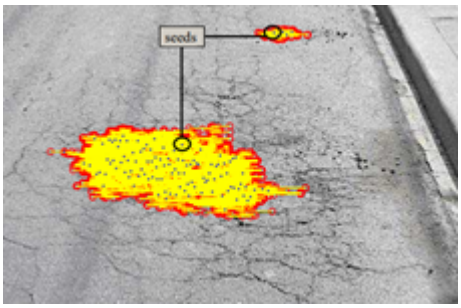
2.3. ábra: Szűrt bináris kép [2]

szöbértékből kivonták a kép átlagos intenzitását, majd ennek az abszolút értékét kettővel megszorozták. A delta értéket arra használták, hogy az eredeti szürkeárnyalatos képből két új vágott képet hozzanak létre. Az eredeti kép aljáról delta sort vágnak le, ez lett az egyik vágott kép, aztán eredeti képnek a tetejéről vágtak le delta sort, ez lett a másik. Majd ebből a két vágott kép különbségéből határozták meg a bináris képet. Az eredmény sokkal tisztább, jobban kiemeli a kátyút, és a zajok nagyrészét kiszűrte a képről. A 2.2. ábra jól rámutat, hogy a kátyú mellett leginkább egyenes vonalak maradtak meg. Ezeket excentricitás módszer segítségével távolítják el, amit ismertettem a Pothole detection in asphalt pavement images című alfejezetben a 4.oldalon.Továbbá a nagyon apró régiókat, alakzatokat is törlik, végül minden olyan alakzatot törölnek, ami hozzá ér a kép széléhez. Így kapnak egy szűrt csak a kátyút tartalmazó bináris képet.

A cikkben ezzel párhuzamosan elvégzik az eredeti szürkeárnyalatos kép hisztogramján a spektrális klaszterezést. A klaszterezés lényege, hogy a hasonló előfordulású intenzitásokat egy csoportba gyűjtsék ki, így létrejön k-db csoport, klaszter. A hasonlósági vizsgálatot Gauss kernellel teszik, a hisztogram intenzitások(0-255ig) gyakoriságát hasonlítják össze az egyenlet segítségével, az így kapott étékeket a hasonlósági mátrixban tárolják el. Aztán fokszám majd Laplace mátrixot számolnak, a fokszám mátrix esetén végig iterálnak a hasonlósági mátrix sorain, és szummázzak azokat ezzel egyfajta összegzett népszerűség értéket rendelnek az intenzitásokhoz. Majd egy új mátrix főátlóiba beírják ezeket a népszerűség értékeket. Laplace mátrix számítása, a fokszám mátrixból kivonják a hasonlósági mátrixot, az így kapott 256×256-os mátrix főátlója az intenzitások fokszámait, míg a többi elem a gráf szerkezetét írja le. Ezután ennek a Laplace mátrixnak elvégzik az eigen-decomposition-t (sajátérték-felbontását), ami kiszámítja a sajátértékeket. A cikkben említett algoritmus 1 ezeket sajátértékeket használja fel, hogy automatikusan meghatározza a klaszterek számát (k).

Végezetül, a tisztított bináris képet, valamint az eredeti szürkeárnyalatos kép klaszterezéséből kapott eredményt felhasználva, az algoritmus 2-őt futtatják, ami seed points-okat (mag pontokat) hoz létre, ezekkel a pontokkal összekapcsolja a kétféleképpen feldolgozott képet. A 2.4. ábra szemlélteti, a mag pontokat és az összekapcsolt végeredményt.

[7]



2.4. ábra: A cikkben bemutatott módszer végeredménye [2]

A cikkben leírt módszer fő előnye, hogy egy olcsó, felügyelet nélküli eljárás, amely nem igényel drága berendezéseket vagy külön tanítási fázist. A spektrális klaszterezés alkalmazásával a tesztadatbázison „81%-os pontosságot” értek el, ezen felül „rough estimation” (durva becslést) adtak a kátyúk területére [7]. Ilyesfajta durva becslés jól jöhet a későbbiekben, amikor a kátyú méretét és elhelyezkedését fogom vizsgálni. Hátránya azonban, hogy ez a becslés valóban csak közelítő, mivel a végső területet egy egyszerű téglalap alapján számolja ki, de viszonyítási alapnak jó lehet. A cikk nem tárgyalja a valós idejű feldolgozást, ami részben érthető, hiszen a hibás képkockák kiválasztása manuálisan végzik. Bár a teszteléshez használt hardver alapján „Intel Core2 2.80GHz CPU és 4GB RAM” a valós idejűség akkoriban nem volt reális cél, modern számítási kapacitással ez ma már megoldható lenne [7]. A valós idejű működés eléréséhez azonban elengedhetetlen a teljes folyamat automatizálása, kezdve egy olyan algoritmussal, amely képes a videofolyamból pontosan, emberi beavatkozás nélkül kiszűrni a releváns, hibát(kátyút) tartalmazó képkockákat.

2.1.3 Pothole detection on asphalt pavements from 2D-colour pothole images using fuzzy *c*-means clustering and morphological reconstruction

Ouma és Hahn 2017-ben egy többlépcsős, hibrid algoritmust dolgoztak ki. Ennek oka, hogy úgy vélték a 2D képfeldolgozó módszereknek alapvető korlátai vannak kátyúdetektálás terén. Különösen kiemelték, hogy a széles körben alkalmazott küszöbölési eljárások gyakran érzékenyek a változó fényviszonyokra és az útburkolat textúrájának inhomogenitásaira. Ezek a tényezők rossz irányba befolyásolják a szegmentálás kimenetét. Illetve, a hagyományos, "kemény" szegmentálási módszerek rugalmatlanságáról esett szó a cikkben, amelyek a kép minden egyes pixelét egyértelműen és kizárólagosan besorolják egyetlen kategóriába.

A javasolt megoldásban az első lépés a zajszűrés és a szegmentációhoz használatos küszöbérték megtalálás áll. Ehhez A wavelet transzformációt használnak, a fő előnye,

hogy több skálán elemzi a képet egyszerre, ellentétben a hagyományos szűrőkkel. Az-az képes a kép finomabb és durvább részleteit külön kezelni. Ezáltal a zaj jelentős részét eltünteti miközben a fontos jellemzőket jobban megtartja. Cikkben a Daubechies waveletet-et használnak, felbontják a képet különböző szintekre, szétválasztva a durvább, kisebb felbontású pixel információkat, és a magasabb, finomabb, részletinformációkat. Ezt követően a részlet koefficienseken "lágy" küszöbölést végeznek, amely a zajnak tulajdonított kis értékű koefficienseket nullázza vagy csökkenti, miközben a fontosabbakat megőrzi. Végül az inverz wavelet transzformációt segítségével a módosított koefficiensekből visszaállítják a képet, amely így egy simított, zajcsökkentett lesz.

Klaszterezés lépésben a Fuzzy c-means clustering-et használják a szerzők. Mert így bizonyos pixeleket egyszerre több kalszterben is számon tudnak tartani százalékos arányba kifejezve (például egy pixel 70%-ban kátyú, 30%-ban út). Az algoritmus véletlenszerű kezdeti klaszterközéppontokból indul ki majd felváltva frissíti a pixelek tagsági fokait a középpontoktól való távolság alapján. Aztán újra számolja a klaszterközéppontokat a pixelek tagsági fokokkal súlyozott átlagaként, amíg a tagsági fokok változása egy küszöbérték alá nem esik. Ez a folyamat a költségfüggvényt minimalizálja, amely a pixelek és a hozzájuk tartozó klaszterközéppontok közötti, tagsági fokokkal súlyozott távolságok összegét méri.

Utolsó lépésben a morfológiai rekonstrukciót hajtanak végre, amely pontosítja a kalszterezést. Két képet használ: egy marker képet, ami a klaszterezés során azonosított potenciális kátyú pixeleket tartalmazza, és egy maszk képet, ami az eredeti wavelet-szűrt kép intenzitásait használja, felső korlátként. A két bement alapján újra felépíti a képet a markerek alapján, de csak azokat a részeit tartja meg, amelyeket a maszk "megenged". [8]

Ez a hibrid módszer kecsegtető és érdekes részekből épül fel. Az elért „87.5%-os átlagos pontosság” és az hogy nem igényel tanító adatbázist, nagyon jó eredmény [8]. Ugyanakkor jelentős hátrányai is vannak, amelyek korlátozzák a gyakorlati alkalmazhatóságát. A legkritikusabb probléma a rendkívül lassú, „95 másodperces átlagos futási idő képenként”, ami teljesen kizárja, hogy egy az egyben fel tudjam használni ezt a megvalósítást [8]. Bár figyelembe véve a szűrés részletességét, a klaszterezést, majd pedig az iteratív morfológiai rekonstrukciót, ez érthető futási idő még hardvertől függetlenül is. Az a baj ezzel a megoldással, hogy a Fuzzy c-means clustering-et megelőzően nagyon pontos szűrésre van szükség, ami viszont számításigényes. Ha pedig nem megfelelő a szűrés akkor a szegmentálás nem lesz megfelelő. Tehát csak a morfológiai rekonstrukción lehetne drasztikusan változtatni, ám azt nem hiszem, hogy olyan sokat számítana, és hirtelen valós időben futtathatóvá válna a program.

2.1.4 Watershed-alapú valós idejű képfeldolgozás aszfaltúton található többszörös kátyúk észlelésére

Tran Duc Chung és M. K. A. Ahamed Khan olyan valós idejű megvalósítást javasolnak 2019-ben, ahol a feldolgozás egyszerűsége és a feldolgozási ideje áll a középpontban. Ezt úgy érik el, hogy vesznek egy RGB (red (vörös), green (zöld) és blue (kék)) képet, ezt szűrek árnyalatos képpé konvertálják majd kijelölnek egy ROI (Range of Interest (érdekes régió))-t a képen belül, amit vizsgálni fognak. Ezt követően csak a ROI-val fognak számításokat végezni, ezzel csökkentve a számítási kapacitást és időt. A ROI-n belül bináris és Otsu féle küszöbölést alkalmaznak, hogy a szegmentálást, azaz a hátteret az objektumtól el tudják jól különíteni. Következik a zajszűrés a ROI-n majd a megmaradt összefüggő objektumok kiemelése dilatációval, erre két külön kernelt használnak. A dilatációval eltűnnek a fekete (háttér) szigetek a ROI-n belül és csak azok maradnak feketék, amik biztosan a hátteret képzik. Ezt követően távolságtranszformációt hajtanak végre a markerek (jelölők) kiszámításához. A távolságtranszformációval minden egyes fehér pixelre kiszámolják, hogy milyen távol van a legközelebbi fekete pixeltől ezzel minden fehér pixelnek egy magasságot ad. Ezzel a módszerrel meghatározhatóak olyan markerek, mint biztos háttér és biztos előtér (objektum), illetve ismeretlen régió. Végül a marker térképet felhasználva az eredeti RGB képen meghívják a watershed (vízgyűjtő) algoritmus, ami az ismeretlen régiókat elválasztja hátérré vagy előtérré. [9]

Nagyon inspirálónak tartom a ROI használatát, mert az általános, hogy a képeken több út (háttér) szerepel, mint kátyú (objektum), ROI meghatározással sok felesleges számítást lehet megspórolni. A cikkben vannak pontos mérési adatok FPS (Frames Per Second (képkocka másodpercenként))-ben megadva, ebből az vonható le következtésként, hogy minél több kátyú vagy kátyú részlet van a képen, annál rosszabb a feldolgozás sebessége, legrosszabb mérés 26,4 FPS a legjobb pedig 37,4 FPS [3]. A „képek 640×480 pixel felbontásúak” voltak és az „algoritmus Intel Core i5 5255u processzorral és 4GB DDR3 memóriával ellátott számítógép futatta, 1600 MHz-es buszon”, GPU-t nem említenek [9]. Itt felmerült bennem az a lehetőség, hogy GPU segítségével lehetne gyorsítani azokat az eseteket, amikor több kátyú van egyszerre a képen. Kátyú száma ROI kellene létrehozni, és azok párhuzamosan GPU-n vagy akár CPU-n is futhatnának. A 30 FPS és a feletti teljesítmény meglehetősen jónak mondható és már tényleg valós idejű rendszerként használható lehet.

Pontosságról nem esik szó a cikkben, de feltételezhető, hogy minden úton lévő nagyobb objektumot hasonlóan detektál gondolok itt árnyékra, felfestésre, foltokra tehát nem csak kátyúkat.

Még az Otsu féle statisztikai technikára térnék ki, mert jó minőségű képeken, amin kevés zaj van azokon a küszöbérték meghatározása jó működik jellemzően ilyenek voltak

a példaképek a cikkben. Az objektum mérete, az objektum helyzete és gyengébb, zajos képminőség befolyásolja a teljesítményt és előfeldolgozást kell végezni a képen a jó végeredmény érdekében. [10]

2.2 Konvolúciós neurális hálózatokon alapuló úthibadetektálás

Az előző Klasszikus képfeldolgozáson alapuló úthibadetektálás című fejezetben, általam bemutatott algoritmusok „műveleteket hajtanak végre a bemeneti képeken, hogy hasznos információkat nyerjenek ki belőlük" [11]. Ezek a képfeldolgozó algoritmusok, több egymásra épülő merev lépésekből állnak (előfeldolgozás, szegmentálás, jellemzőkinyerés, osztályozás). Bár a teljesítményük 90% körül mozog, de a számítási idő magasnak mondható, főleg valós idejű rendszerhez viszonyítva. Továbbá, a klasszikus módszerek implementálása során a paramétereket (például: szűrőméretek, küszöbértékek) manuálisan finom hangoljak, hogy az adott bemeneti adathalmazon optimális megoldás elérése érdekében. Ennek következménye az alacsony robusztusság, mivel a végeredmény túlságosan érzékennyé válik a bemeneti adatok specifikus jellemzőire, és az azokhoz igazított paraméterekre. Ez a megközelítés működőképes, de inkább homogén környezetben (például: gyártósori minőségellenőrzés), míg az inhomogén esetekben, mint az úthibák, kátyúk detektálása, ahol az út felülete és a fényviszonyok képkockáról képkockára változhatnak, nem beszélve a kátyúk eltérő tulajdonságairól, ez a módszer nem bizonyul járható útnak számomra.

Deep learning (mélytanulás) a mesterséges intelligencia egyik kulcsfontosságú és napjainkban divatos részhalmaza. Ezen a területen belül a convolutional neural network (CNN, konvolúciós neurális hálózat) jelentik azt a specializált architektúrát, amelyet kifejezetten a képekhez hasonló, rácsszerkezetű adatok feldolgozására és elemzésére fejlesztettek ki. A hálózat három rétegből tevődik össze, konvolúciós réteg, ami a bementi kép vagy a future map (jellemző térkép) pixel értékein halad végig egy vagy több kernel-lel (filter, csúszó ablak) és elvégzi a konvolúció műveletet. Az így létrejött jellemző térkép(ek) a csúszó ablak(ok) által kiemelt tulajdonságokat tartalmazza. Ezt követően a pooling layer (tömörítő réteg) a jellemző térképen szereplő tulajdonságokat felerősíti, és tovább csökkenti a dimenzióméretet. Harmadik a fully connected layer (teljesen összekapcsolt réteg) a képek osztályozást végzi. Az aktivációs függvények teszik képessé a hálózatot arra, hogy az adathalmazban rejlő komplex, nem lineáris mintákat, összefüggéseket felismerjék. A csúszó ablakként funkcionáló konvolúciós kernelek súlyparamétereit a hálózat a tanítási folyamat során (például: backpropagation) finom hangolja. Ez az optimalizálás biztosítja, hogy a hálózat által kinyert jellemzők a kívánt osztályozási eredményhez vezessenek, minimalizálva a hibát a tanító adathalmazon. [11]

A konvolúciós neurális hálózatokon belül három számítógépes látása feladat típust kü-

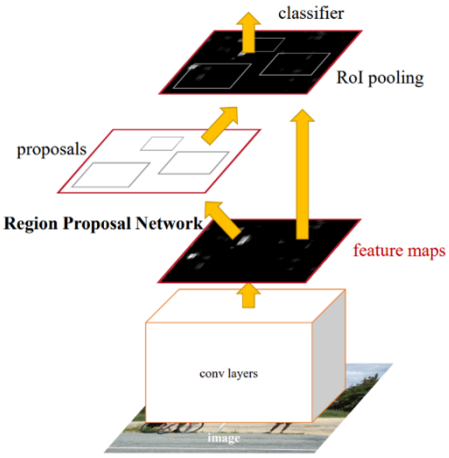
lönböztetünk meg, image classification (kép osztályozás), object detection (objektum detektálás) és semantic segmentation (szemantikus szegmentálás) [11]. A kép osztályozás esetén a kimenet egy címke lesz, például a kép tartalmaz kátyút, vagy a kép nem tartalmaz kátyút. Tehát ebben az esetben a képen szereplő kátyúk darabszámáról, és a betöltött pozíciójukról, sem ad információt. Az objektum detektálás estén ez nincs így, itt a kimenet mind egyes kátyú köré bounding box-ot rajzol, ezzel az összes kátyút és azok pozícióját is jelöli a képen. A szemantikus szegmentálás szolgáltatja a legrészletesebb információt a képről, minden egyes pixelhez címkét rendel. Kijelenthető, hogy jelenleg valós idejű kátyú detektálásra, az objektum detektálás típus használható a legjobban, ezért a továbbiakban csak ezeket fogom részletezni. [12]

2.2.1 Kétfázisú objektumdetektálás Faster R-CNN

2016 január 6.-án Shaoqing Ren, Kaiming He, Ross Girshick, és Jian Sun publikálták a Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks című folyóiratcikket. Abban az időben a legjobb objektumdetektáló algoritmusok a Region-based CNN (R-CNN) családba tartoztak. A korábbi verziók (R-CNN, Fast R-CNN) pontossága már nagyon jónak számított, de a sebességükről ugyan ez nem volt elmondható. Működésük, első lépéseként egy külső region proposals (régiójavaslatok) algoritmus több potenciális bounding box-ot generál. Ezt követően a rendszer egy konvolúciós neurális hálózatot futtat le minden egyes javasolt régión, hogy azokból features (jellemzőket) nyerjenek ki, amelyeket aztán egy különálló osztályozó értékel. A szerzők a sebességen akartak változtatni, a szűk keresztmetszetet a régiójavaslatok jelentették. A régiójavaslatok meghatározásához klasszikus képfeldolgozó algoritmusokat használtak (selective search, edge boxes). A probléma az volt, hogy a javaslatok generálása sokszor annyi ideig tartott, mint maga az objektum detektáló hálózat. A cikk fő áttörése, hogy egy kicsi, gyors neurális hálózattal, Region Proposal Network-el (RPN) hajtják végre a régiójavaslatok generálását.

A következő 2.5. ábra rámutat, hogy a szerzők által leírt RPN jelentőségére, és, hogy miért nevezhető two-stage-nek (két fázisú) az összes R-CNN-en alapuló implementáció. Első fázis a régiójavaslatokért felelős, ez az, amit a RPN hálózat hajt végre. Egyetlen feladata, hogy a teljes feature map-et (jellemzőtérkép) gyorsan végig pásztázza, és olyan régió dobozokat javasol, ahol valószínűleg valamilyen objektum található. Ez a fázis nem mondja meg, hogy pontosan mi van abban a régió dobozban, csak azt, hogy ezeken a területeket érdemes tovább vizsgálódni. Ezt követi a második fázis, ez maga a detektálásért felelős, classifier modul. A RoI pooling elvégzi az összevonást, RPN hálózat által tett javaslatokon és az azokhoz tartozó jellemzőtérkép részleteken, majd az összevonást követően megkezdődik az osztályozást. Eldönti, hogy a dobozban kátyú, autó vagy háttér van. Továbbá, finom hangolja a dobozok koordinátáit, hogy az jobban illeszkedjen az

objektumra. Tehát a folyamat két egymásra épülő lépéből áll, első a javaslat, második az osztályozás, ezért hívják kétfázisú detektornak. [13]



2.5. ábra: Faster R-CNN felépítése [13]

Fontos, hogy itt a RPN nem a nyers bementi képen, hanem a törzs kimentén, azaz a konvolúciós rétegek végső, mély jellemzőtérképén dolgozik. Az RPN a cikkben egy 3×3-as csúszó ablakkal dolgozza fel a jellemzőtérképet. A jellemzőtérkép minden egyes pozíciójában k darab különböző méretű (például: kicsi, közepes, nagy), előre definiált anchor box (horgonydoboz, referencia dobozt) használ. Ezek a dobozok, keretek ugyanott vannak központosítva. Tehát a referencia doboz a 3×3-as csúszó ablak középpontjához vannak társítva, de különböző méretekkel és oldalarányokkal rendelkeznek. Majd a kimenetet egy alacsonyabb dimenziójú vektorrá alakítják. Ezt követően, párhuzamosan két fully connected layer (teljesen összekötött réteg) bemenetére adják ezt a kilapított vektort. Az egyik az objectness score valószínűséget jósolja, azaz, hogy az adott helyen van e bármilyen objektum, vagy csak háttér található azon a területen. Másik a megfelelő horgonydobozt próbálja kiválasztani az objektumhoz.

A Faster R-CNN legfőbb előnye a korábbi kétfázisú detektorokhoz (mint az R-CNN vagy Fast R-CNN) képest elért drámai sebességnövekedés, amellyel megközelíti a valós idejű működést (5-17 FPS). Bár a Watershed-alapú valós idejű képfeldolgozás aszfaltúton található többszörös kátyúk észlelésére című alfejezetben a 10. oldalon ismertette watershed-alapú implementáció jobb, 30 FPS sebességgel működik. A Faster R-CNN kiváló pontossággal detektálja az objektumokat, mivel az RPN jobb minőségű javaslatokat ad, mint a fix algoritmusok. A rendszer hátránya ugyanakkor a tanítási folyamat bonyolultsága, a megosztott jellemzők optimalizálása a cikkben egy komplex, 4 lépéses alternating training (váltott tanítási) sémát követ. Habár általánosságban nézve gyorsnak számít, de ahogy a publikáció címe is jelzi ez "csak" valós idejű megvalósítás felé vezető megoldás. [13]

2.2.2 Egyfázisú objektumdetektálás You Only Look Once v1

2016 május 9.-én jelent meg a Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi közösen publikálták a You Only Look Once (YOLO) Unified, Real-Time Object Detection (az első YOLO verzió). Fő szempont az volt, hogy korábbi komplex, lassú és nehezen optimalizálható megoldásoknál, gyorsabb architektúrát dolgozzanak ki, ami jobban tudja utánozni az emberi vizuális rendszer, gyorsaságát és pontosságát.

A YOLO regressziós problémaként fogja fel a detektálást. Nincs szükség komplex, többlépcsős folyamatokra. Egyetlen konvolúciós neurális hálózatot használnak, és „a rendszernek csak egyszer kell ránéznie a bementi képre, hogy prediktálni tudja a képen az objektumokat és a hozzájuk tartozó pozíciót”. [14]

Első lépésben a bementi 448×448-as felbontású képet felosztja S×S méretű grid-ekre (rácsokra, cellákra), ha egy objektum a középpontja belesik az adott rácsba, akkor az a bizonyos rács felelős, annak az objektumnak a detektálásáért. Fontos, hogy egy objektum átlóghat több különböző cellába is, de a középpontja csak egy cellában lehet, és az a cella felel azért az objektumért. A predikció során minden egyes objektum köré bounding box-okat (határoló dobozokat) rajzolnak, azaz bekeretezi az objektumok területeit a képen belül. Minden egyes határoló doboz az alábbi értékekből épül fel, cellán belüli lokális x,y koordináták, amik a detektált objektum középpontját írják le (ez a cella felel az objektumért). Továbbá tartalmaz w,h globális koordinátákat a teljes képhez képest, ezen információk alapján rajzolják ki a keretet az objektum köré. Fontos megjegyezni, hogy x,y,w,h mind relatív értékek, azaz 0-1 közöttiek így a tanítás kis számokkal zajlik. Minden bekeretezett objektumokhoz még egy confidence score-t (megbízhatósági pontszámot) rendel, amivel mérni tudják, hogy mennyire biztos a modell abban, hogy a bekeretezett doboz tartalmaz objektumot és hogy mennyire pontos a bekeretezése. Egy cellán belül B-db bounding box javaslat jön létre, a javaslatok nem tudják, hogy pontosan mi az objektum (például: kátyú vagy macska), csak az objektum létét és helyét becslik. Van C-db osztály, amit detektálni akarnak, ehhez a cikkben a Pascal VOC adathalmazt használják, amely 20 különböző osztályt tartalmaz (C = 20). Minden egyes cellához a bounding box-októl függetlenül, C-db valószínűséget számolnak a cella vizuális tartalma alapján. Tehát minden egyes cellához tartozik egydarab 20 elemű one hot vector, ami tartalmazza a cellához tartozó osztály valószínűségeket. Végül ezeket a predikciókat egy tesorba (többdimenziós adattároló) helyezi, aminek a dimenzió szám képlete a következő: [14]

„$S \times S \times (B \times 5 + C)$”[14]

Ez a tensor a hálózat feje, adja meg a hálózat végső kimenetét, a cikkben B=2-vel (egy cellában két bounding box javaslat jön létre), de a kimenetben egy cellához csak az

egyik javaslatot tartják meg. Ennek menet a következő, a cellához tartozó maximális C objektum valószínűséget, össze szorozzák külön, külön mind a két javasolt bounding box-hoz tartozó confidence score-ral, és a magasabb szorzat eredmény dönt. Végül a relatív kimenti értékeket, abszolút pixel koordinátákká kell visszaszámolni a kirajzoláshoz. Így áll elő, a bementi 480×480-as képen, hogy hol és milyen objektumok találhatóak, ezeket az objektumokat a pixel koordináták alapján bounding box-okkal jelölik körbe, a kimeneti képen.

A teljes YOLO architektúra két részre osztható, backbone (törzs) és a head (fej), előbbiekben szó volt a hálózat végső kimentéről ez a hálózat fejéhez tartozik. A hálózat törzse 24 darab konvolúciós rétegből áll, ezek a rétegek között pooling rétegek helyezkednek el a kép térbeli méretének csökkentése érdekében. Az architektúrának a törzs része felelős a képi információk kinyeréséért és azok megtanulásáért. Ezzel szemben a fejrész kettő fully connected layer-ből áll, a bemente egy kilapított one hot vector a kimenete pedig az előző bekezdésben részletezett tensor. A fejrész felelős a végeredmény előállításáért a kinyert jellemzők alapján. A YOLO egy egységes hálózat, amely a teljes detektálási feladatot ellátja, de a hálózat betanítása két egymásra épülő részből áll. Első a pretraining (előtanítás) ebben a fázisban a hálózat 20 konvolúciós réteget, egy average pooling réteget. végül egy fully connected réteget tartalmaz. A tanításhoz, az ImageNet adathalmazt használják, a tanítás végén az alapvető képi, vizuális jellemzőket megtanulja a hálózat, így képes lesz a képeket osztályozni. Ezután a betanított törzs hálózatot átalakítják, hogy a képek klasszifikációján túl, objektum detektálásra is képes legyen. Az előtanított 20 konvolúciós réteg után további 4 új, véletlenszerűen inicializált konvolúciós réteget és 2 új, szintén véletlenszerűen inicializált fully connected réteggel létrejön a YOLO architektúra. Míg az előtanítás 224×224-es bementi felbontású képeken zajlott, addig a detektáláshoz részletesebb vizuális információra van szükség, ezért a tanítása következő része 480×480-as növelt képeken folytatódik. Itt már a Pascal VOC adathalmazt használják fel a tanításhoz, és egytelen komplex loss function-t (hibafüggvényt) készítettek hozzá a szerzők. A tanítás során azokat a cellák tartják fontosabbak, amik objektumot tartalmaznak, és azok a cellák kevésbé fontosnak, amik nem tartalmaznak objektumot. Azért lényeges ez, mert egy objektumot egy cella reprezentál (az a cella, ahol az objektum közepe található) így általánosságban több olyan cella van, ami nem tartalmaz objektumot. Ezen a logika mentén másképp kell "büntetni" a cellákat tanulás során, ha azok felelősek egy objektum detektálásáért, akkor 5 szeresére megnövelik a bounding box koordinátáinak pontosságáért járó "büntetést", ha viszont csupán hátteret tartalmaz az adott cella, akkor 0.5 szeresére csökkentik az objektumot nem tartalmazó cella "büntetését". Így teremtenek egyensúlyt a tanítás során az eltérő cellák és azok eltérő előfordulása között.

Összességében a YOLO (You Only Look Once) egy forradalmian új, egységesített objektumdetektálási megközelítés, amely szakított a korábbi, lassú és többlépcsős (pl.

R-CNN-féle) módszerekkel. Ennek az egységes architektúrának a meghatározó előnye az extrém sebesség, amellyel megteremtette a valós idejű objektum detektálást. További előnye, hogy a hálózat a teljes képet látja, így globális kontextussal dolgozik, ezért sokkal kevesebb téves pozitív (háttér) hibát vét, mint a korábbi régió alapú módszerek, és jobban generalizál (általánosít). A felsorolt tulajdonságok miatt nagy valószínűséggel ezt fogom használni a kátyúk detektálásához. Ugyanakkor a módszer hátránya a pontosság terén fellelhető, mivel a YOLO több lokalizációs hibát vét, és nehezebben birkózik meg a kis méretű objektumok pontos helymeghatározásával. [14]

2016-óta számos YOLO verzió jelent meg, amelyek fejlesztése már nem feltétlenül az előbb bemutatott első verzió szerzőihez kötődik, hanem több különböző, független kutatócsoport, fejlesztő és cég (például az Ultralytics) nevéhez. Ez a fajta fejlesztői sokszínűség párhuzamosan zajlott a modellcsalád folyamatos architekturális evolúciójával. Míg a korai verziók viszonylag egyszerű, elemekből építkeztek addig, az újabb (mint a YOLOv4, v5, v7 vagy v8) már szignifikáns változtatásokon mentek keresztül, beleértve a gerinc hálózatok (pl. CSPDarknet) optimalizálását, a jellemző-aggregációt végző nyak modulok (pl. FPN, PANet) bevezetését. Valamint az alapvető detektálási stratégia módosulását, például a Faster R-CNN átvett anchor box, majd az anchor-free (horgonymentes) megközelítések alkalmazását. Ezek az újítások együttesen tették lehetővé a sebesség és a pontosság folyamatos fejlődését [15], [16].

Minden egyes YOLO verziót bemutatása helyett, a következő fejezetben, csak a feladatomhoz specifikusan kapcsolódó verziókra térek ki. Tehát olyan valós idejű, ipari gyakorlatban használatos, erőforrás hatékony, és alacsony számítási igényük verziókra, amelyeket korlátozott erőforrású eszközökön is jó futási eredményt produkálnak.

2.2.3 YOLO metrikák

Először az összehasonlítási alapot adó egységes metrikákat írom le, és ezek mentén fogom összehasonlítani a YOLO verziókat. Van négy darab alapmutató, amiből a különböző mérési metrikákat kell számolni. Ezek a true positive (TP, pedálú: a modell kátyút jósol a kimeneten és valóban van kátyú a képen, a bounding box is jó helyen van, IoU), false positive (FP, pedálú: a modell kátyút jósol, de valóban nincs kátyú a képen, vagy van kátyú a képen és a bounding box rossz helyen van). False negative (FN, pedálú: a modell nem talál kátyút a képen, pedig van kátyú a képen csak nem találta meg), true negative (TN, pedálú: nem talál kátyút a képen mert valójában nincs is kátyú rajta) [17].

Intersection over Union (IoU) ez felelős a hálózat által prediktált bounding box lefedettségéért, azaz a kimenet mennyire van átfedésben a tanító képhez (ezeken a képeken az objektumokat kézzel, ember által bejelölt, határoló dobozok jelölik, Angol kifejezéssel élve "ground truth"-nak nevezhetőek) képest. Képlet: IoU = két boundig box metszetének

a területe / két bounding box uniójának a területe, ha a kimenet 0 akkor nincs átfedés, ha a kimenet 1 akkor tökéletes az átfedés. Küszöbértéket kell meghatározni a IoU-hoz, ha a köszöneteknél nagyobb eredményt ad az IoU képlet akkor az true positive kimenetnek tekinthető, ellenkező esetben false pozitive-nak [15].

Precision (precízió, pontosság) a hálózat kimenetének a megbízhatóságát méri, helyes kimenetek száma / összes kimenet száma, képlete: „P = TP / (TP + FP)” [18].

Recall (felidézés, érzékenység) ez a metrika az alaposságot méri, mennyi kátyút ismer fel a bemeneten, helyesen megtalált objektumok száma / összes objektum a képen, képlet: „R = TP / (TP + FN)”[18].

Average Precision (AP) a Recall-ból és a Precision-ból számítható metrika, ha a precízió küszöbértékét alacsonyra állítjuk akkor sok objektumot fog találni a képen, hátránya, hogy megnő a false pozitvok száma. Ennek köszönhetően magas lesz az érzékenységi érték. Ellenkező esetben, ha a precízió küszöbértékét magasra állítjuk, akkor csak azokat az objektumokat észleli, amiben nagyon biztos, és sokat figyelmen kívül fog hagyni, azaz alacsony lesz az érzékenység mutató. Tehát a Precision és a Recall függenek egymástól és ez oda vissza igaz. Az a AP-t objektumkategória specifikusan számolják külön, külön. Ehhez kirajzolják a Precision-Recall görbét, majd a görbe alatti területet számolnak, tanító adathalmaztól (Pascal VOC vagy Microsoft COCO) függően kicsit eltérő lépésekkel.

Ezzel szemben a mean Average Precision (mAP) az összes objektumkategória átlagát számítja ki, egyetlen számot ad vissza, amit a modell általános teljesítményének tekintenek [15], [17].

2.2.4 YOLOv5, YOLOv8 és YOLOv10 fejlődése

Az előző YOLO metrikák pontra támaszkodva és a [19]-forrás alapján hasonlítom össze a címben említett YOLO verziókat. Ebben az összehasonlításban csak olyan verziókról, és azok specifikus változatairól esik szó, amelyek kifejezetten erőforrás hatékonyan tudnak működni, valós időben, „edge deployment” [19] (peremhálózati eszköz, például: okostelefon, drón, autóba szerelt vezérlőegység) környezetben.

A YOLOv5, YOLOv8 és YOLOv10 verziókon belül, megkülönbözettünk további változatokat, mint nano(n), small(s), medium(m), large(l), extra large(x). Ezek a megnevezések a modell méretére, és a hozzá tartozó számítási igényére utalnak. Tehát minél nagyobb a modell, annál több paramétert tartalmaz, ezáltal nagyobb számítási igénnyel rendelkezik, megnő a futási idő, cserébe egyre jobb a modell általános teljesítménye (mAP).

Az Ultralytics 2020-ban mutatta be a YOLOv5-öt, ami jelentős ugrást hozott teljesítmény és a használhatóság terén is, így gyorsan nagy népszerűségre tett szert. Az 2.1.

táblázat szemlélteti az egyes modell méretek közötti különbségeket. A méréseket a Microsoft COCO adathalmazon végezték [19].

2.1. táblázat: YOLOv5 verziók összehasonlítása [19]

Modell	Bemeneti kép mérete	AP	AP 50	CPU futási idő (ms)	Paraméter szám (M)	FLOPs (B)
YOLOv5n	640	28,0%	45,7%	45	1,9	4,5
YOLOv5s	640	37,4%	56,8%	98	7,2	16,5
YOLOv5m	640	45,4%	64,1%	224	21,2	49,0
YOLOv5l	640	49,0%	67,3%	430	46,5	109,1
YOLOv5x	640	50,7%	68,9%	766	86,7	205,7

Jól látható, hogy minél nagyobb a paraméterszám annál pontosabb a rendszer. Illetve az is, hogy minél kisebb a paraméterszám annál jobb a futási idő. Számomra a futás idő kritikusnak tekinthető, ezért olyan kompromisszumot kell találnom, ahol kellő gyorsasághoz elfogadható pontosság társul.

2023-ban megjelent a YOLOv8 ugyancsak az Ultralytics csapat által, ami a YOLOv5 sikeres alapjaira épített, ennek a tovább fejlesztésnek az lett az eredménye, hogy jobb átlagos teljesítmény mutatókkal rendelkezik, mint elődjei [19]. A modell gyorsítása mögött pedig az NVIDIA TensorRT optimalizációs szoftver (SDK) áll. A TensorRT kifejezetten mély neurális hálózatok optimalizálására fejlesztették ki, NVIDIA GPU-khoz. Ezzel segítve azokat a valós idejű rendszerek működését, ahol a sebesség és a teljesítmény rendkívül fontos. Ezt kvantálással érik el, a mély neurális hálózatokat általában 32-bites lebegőpontos (FP32) formátumban tanítják, a minél pontosabb végeredmény elérése érdekében. A tanítás befejezte után hajtják végre a kvantálást INT8-ra, TensorRT SDK segítségével az adott NVIDIA GPU-ra. Előnye, hogy a modell mérete a negyedére csökken, ezáltal a sebességnövekedés extrémnek mondható, cserébe viszont pontosság vesztés lép fel [20] , [21]. A 2.2. táblázat mérési eredményei a teljesítmény mellett, a GPU (NVIDIA A100) futás időbeli hatalmas különbségekre mutat rá, itt is a Microsoft COCO adathalmazon.

2.2. táblázat: YOLOv8 és a kvantálás ereje [19]

Modell	Bementi kép mérete	AP	CPU ONNX futás idő (ms)	A100 TensorRT futás idő (ms)	FLOPs (B)
YOLOv8n	640	37,3%	80,4	0,99	8,7
YOLOv8s	640	44,9%	128,4	1,20	28,6
YOLOv8m	640	50,2%	234,7	1,83	78,9

Fontosnak tartom kiemelni, hogy a 2.2. táblázatban szereplő A100 TensorRT futás idő (ms) oszlop értékei, egy csúcskategóriás hardveren produkált futásidőket mutatja. Az NVIDIA A100 Tensor Core GPU kifejezetten adatközpontok számára lettek fejlesztve, hogy a mesterséges intelligencia modellek valós időben tudjanak működni [22].

YOLOv10 2024-ben jelent meg, Tsinghua Egyetem kutatói álltal. A fő újítása az objektumokhoz tartozó bounding box hozzárendelésben rejlik. Korábbi YOLO verziók one-

to-many hozzárendeléssel dolgoztak, azaz a detektálás során egy objektumhoz több bounding box-ot prediktálnak, így több visszajelzés keletkezik, amiből a tanulás során jobb AP érhető el. Ennek az a hátránya, hogy a redundáns bounding box-ok közül ki kell választani egyet, minden objektumhoz. Ezt egy utófeldolgozási lépéssel, a Non-Maximum Suppression (NMS) algoritmussal oldják meg, ami lassítja a működést. Ezzel szemben a one-to-one hazarendelés szigorúan egy bounding box-ot rendel az objektumokhoz. Ennek előnye, hogy nem kell utófeldolgozó NMS algoritmust használni, de a tanítás lassabbá, és kevésbé pontossá válik. YOLOv10 ötvözi a one-to-one és a one-to-many hozzárendelést, „Consistent Dual Assignments”[23] néven hivatkoznak rá [19]. Ennek az architektúrális változtatásnak a futás béli előnyeit mutatja be a következő 2.3. táblázat.

2.3. táblázat: YOLOv8 és v10 összehasonlítása [19]

Modell	**Paraméter szám (M)**	**FLOPs (B)**	**AP**	**End-to-end futásidő (ms)**	**Futásidő (ms)**
YOLOv8n	3,2	8,7	37,3%	6,16	1,77
YOLOv10n	2,3	6,7	39,5%	1,84	1,79
YOLOv8s	11,2	28,6	44,9%	7,07	2,33
YOLOv10s	7,2	21,6	46,8%	2,49	2,39

A 2.3. táblázatban szereplő end-to-end futás idő, a teljes végrehajtási időt mutatja meg, azaz a gyakorlati detektálási időt. Míg a futás idő (ms) oszlop csak a neurális hálózat számítását méri, ebből jól látható az utófeldolgozás időbeli plusz költségének mértéke. A méréseket NVIDIA T4 GPU-n végezték FP16-os kvantálással, a Microsoft COCO adathalmazon [23]. Ebből következik, hogy eltérő mérési értékek szerepelnek, 2.2. táblázat és a 2.3. táblázat között, YOLOv8 verziók terén.

2.2.5 Rendelkezésre álló nyílt adathalmazok és versenyek a témában

A projektem adatkészlet kiválasztásának szempontrendszere a következő. A fejlesztés fókuszában a valós idejű, járműbe szerelt dashcam (fedélzeti kamera) videofolyamán alapuló kátyúdetektálás áll. Ebből adódóan olyan adathalmazra van szükségem, amely kifejezetten ebből nézőpontból rögzített, valós forgalmi szituációkat tartalmazó képekből áll.

A szakirodalomban elérhető számos, úthibákkal és autonóm vezetéssel foglalkozó globális referenciabázis, azonban a feladat fókusza miatt ezek jelentős része nem alkalmazható közvetlenül a céljaim eléréséhez.

Az útfenntartási gyakorlatban széles körben elterjedtek a speciális mérőjárművekkel vagy drónokkal, top-down (felülnézetből) rögzített adathalmazok, mint például a German Asphalt Pavement Distress (GAPs) [23] vagy a Crack500 [24]. Ezek az adatbázisok kiemelkedő, pixel szintű pontosságot nyújtanak az aszfalt textúrájának és repedéseinek

vizsgálatához. De a merőleges képeken nem tudom felhasználni mert más perspektívából közelítik meg a problémát.

Az adathalmazok másik jelentős szegmensét az önvezető járművek számára készített globális, nagy diverzitású (időjárás, napszak) referenciabázisok adják, mint amilyen a Cityscapes [25] vagy a BDD100K [26]. Ezek az adathalmazok kiválóak a szemantikus szegmentációra (például az útfelület, a sávok, a gyalogosok és a járművek elkülönítésére). Ugyanakkor nem fókuszálnak a lokális úthibákra: nem tartalmaznak dedikált, bounding box ellátott jelöléseket kátyúkra vonatkozóan.

A projekthez leginkább illeszkedő, okostelefonos és fedélzeti kamerás felvételekből építkező Road Damage Dataset (RDD) adathalmazokra, illetve az ezekhez kapcsolódó globális versenyek (pl. ORDDC) eredményeire fogok támaszkodik. [27]

Maeda és munkatársai a 2018-ban publikált kutatásukban a világon elsőként hoztak létre egy olyan nagyméretű, publikus adathalmazt, amely kifejezetten a mozgó járművekből, okostelefonnal vagy fedélzeti kamerával rögzített úthibákra fókuszál. A közzétett adatbázis elsődleges célja egy olyan egységes referenciabázis megteremtése volt a kutatók számára, amely segítségével a különböző úthibákat detektáló algoritmusok teljesítménye objektíven összehasonlíthatóvá váljék.

Az „adathalmaz összesen 9 053 darab, valós forgalmi szituációban készült képet tartalmaz, amelyeken 15 435 útkárosodást jelöltek meg bounding box-okkal”[28]. A szerzők a hibákat nyolc különböző osztályba sorolták, egy hierarchikus rendszer alapján. A sérüléseket első lépésben repedésekre és egyéb hibákra bontották. A repedéseket ezt követően finomabb kategóriákra osztották tovább. Az egyéb károsodások kategóriája a kátyúk mellett olyan specifikus problémákat is magában foglal, mint például az útburkolati jelek kopása vagy elmosódása.

A publikációban a szerzők kiemelik, hogy a gépi látás és a képfeldolgozás területén korábban egyetlen kutatás sem fedte le az útkárosodások ilyen széles és részletes skáláját. A 600×600 pixel felbontású felvételeket rendkívül változatos földrajzi helyszíneken és eltérő időjárási körülmények között rögzítették, aminek köszönhetően az adatkészlet kiemelkedően jól reprezentálja a valós, mindennapi vezetési környezetet. [28]

A 2018-as kezdeményezést követően a technológia és az adatigény robbanás szerűén megnőtt. Ennek a fejlődésnek a következő nagy mérföldkövét Arya és kutatótársainak 2022-es munkája, a CRDDC (Crowdsensing-based Road Damage Detection Challenge) nevű nemzetközi verseny, valamint az ehhez kapcsolódó RDD2022 adathalmaz publikálása jelentette. A tanulmány rámutat arra, hogy a korábbi rendszerekhez képest több kritikus ponton is paradigmaváltás történt az úthiba-detektálásban.

A legszembetűnőbb változás az adathalmaz mérete és földrajzi diverzitása volt. Míg a

2018-as verzió kizárólag Japánra korlátozódott, addig a 2022-es adatbázis már hat különböző ország (Japán, India, Egyesült Államok, Csehország, Norvégia és Kína) úthálózatát fedi le. A jelentősen kibővített adatkészlet több mint 47 000 valós forgalmi szituációban rögzített képet és mintegy 55 000 bounding box-al ellátott annotációt tartalmaz. Ez a nemzetközi kiterjesztés tudományos szempontból elengedhetetlen volt ahhoz, hogy a modellek általánosító képességét vizsgálni lehessen.

Egy másik jelentős változás a hibakategóriák egyszerűsítése és ipari standardizálása volt. A 2018-as kutatásban használt, nyolc osztályt tartalmazó és gyakran nehezen elkülöníthető kategóriákkal dolgozó rendszert, egy sokkal letisztultabb, négy fő kategóriából álló struktúrára cserélték. Ez az új D-kódos rendszer kizárólag a kritikusan fontos strukturális hibákra fókuszál. A hosszirányú repedésekre (D00), a keresztirányú repedésekre (D10), az aligátorrepedésekre (D20), valamint a kátyúkra (D40). Az olyan kevésbé releváns kategóriákat, mint az útburkolati jelek kopása vagy elmosódása, a hatékonyság érdekében elhagyták.

Végül a tanulmány rávilágított egy egyértelmű technológiai generációváltásra is. Míg az első kutatásoknál még a régebbi SSD (Single Shot MultiBox Detector) architektúrák domináltak, a 2022-es nemzetközi verseny eredményei empirikusan is bizonyították, hogy az egyfázisú detektorok, kiemelten a modern YOLO (You Only Look Once) változatok vették át a vezető szerepet. A YOLO modellek az RDD2022 adathalmazon kimagasló pontosságot és robusztusságot mutattak, végleg ipari sztenderddé válva a fedélzeti kamerás úthiba-detektálás területén. [27]

A következő ORDDC'2024 kihíváson pontosan ugyanazt az RDD2022 nevű adathalmazt használta, amelyet a CRDDC'2022 verseny során tettek közzé. A technológiai újítás a valós idejű működés és a pontosság egyensúlyának megteremtése érdekében a 2024-es év kiemelkedő technológiai trendjévé a tudáspárlat, valamint a könnyűsúlyú neurális architektúrák alkalmazása vált. [29]

2.2.6 Transfer Learning és fine-tuning a gyakorlatban

A gépi tanulás, és különösen a mélytanulás (deep learning) egyik legnagyobb kihívása, hogy a modellek sikeres betanításához hatalmas mennyiségű, címkézett adatra van szükség. Azonban a megfelelő méretű adathalmazok összegyűjtése gyakran túlságosan drága, időigényes, vagy egyszerűen kivitelezhetetlen feladat. Erre a problémára nyújt megoldást a transfer learning (transzfertanulás). Aminek alapelve, hogy egy korábban már megoldott feladat során megszerzett tudást egy új, de kapcsolódó problémára alkalmaznak. A folyamat során a modell egy úgynevezett source domain-ból (forrástartományból) származó információkat, például egy hatalmas, általános képadatbázison betanított hálózat súlyait emel át, egy target domain-ba (céltartományba), ezzel drasztikusan csökkentve az

új feladat adatigényét.[30]

Yosinski és kutatótársai [31] bizonyították, hogy a konvolúciós neurális hálózatok (CNN) rétegei hierarchikusan, az általánostól a specifikus felé haladva építik fel a vizuális tudásukat. A hálózat legelső rétegei univerzális, úgynevezett general jellemzőket tanulnak meg felismerni, mint például a színfoltok, élek, vonalak, sarkok. Ezek a vizuális alapelemek szinte minden képi adathalmazban azonosak, így könnyen átvihető más hálózatokra is. Ezzel szemben a hálózat mélyebb, végső rétegei az eredeti feladatra erősen specifikus jellemzőket detektálnak (például egy adott objektum osztályhoz tartozó jellemzőket).

Erre a megfigyelésre épül a fine-tuning (finomhangolás) technikája. Ahelyett, hogy egy hálózatot a nulláról, véletlenszerűen inicializált súlyokkal kezdenének el tanítani, egy nagy adathalmazon előtanított modell súlyait emelik át kiindulásnak. A finomhangolás során a hálózat első, általános vizuális jellemzőket felismerő rétegeinek súlyait megtartják, rögzítik. Így csak az utolsó, feladatspecifikus rétegeket, valamint az osztályozó réteget tanítják újra a saját, kisebb adathalmazunkon. Ez a megközelítés lehetővé teszi a modell számára, hogy a korábban rögzített álltalános jellemzőkre építve hozzáigazítsa a specifikus rétegeket az új problémához. [31]

A transzfertanulás és a finomhangolás együttes alkalmazásával drasztikus előnyökre lehet szert tenni a nulláról történő betanítással szemben. Zhuang és munkatársai [30] alapján ez három fő teljesítménymutatóban jelentkezik a tanulási görbén.

Magasabb kezdeti teljesítmény, mivel az előtanított modell már az első tanítási lépések előtt felismeri a vizuális alapokat, a kezdeti pontossága sokkal magasabb szintről indul, mint egy inicializálatlan hálózaté.

Gyorsabb konvergencia, a finomhangolás során a hálózat meredekebb tanulási görbét ír le, vagyis nagyságrendekkel kevesebb iteráció és számítási idő alatt éri el az optimális pontosságot.

Legfontosabb hozadék a jobb végső teljesítmény, azaz a végső pontossága szinte mindig felülmúlja a nulláról tanított modellekét, még abban az esetben is, ha az utóbbiakat végtelen ideig próbálják tanítani a rendelkezésre álló korlátozott adathalmazon.

2.3 Detektált hibák lokalizálása

A kátyúk magabiztos felismerését követően, ebben a szakaszban a detektált úthibák pontosabb térbeli pozíciójának meghatározása a cél, amely a képi információk valós világkoordinátákra történő leképezéssel (image to world konverzió) valósítható meg. Jelen fejezet a geometriai képalkotás elméleti hátterét, azt követően pedig a transzformációs eljárásokat tárgyalja, amelyek lehetővé teszik a pixelkoordináták fizikai távolsággá alakí-

tását.

2.3.1 Geometriai képalkotás elméleti háttere

Lyukkamera modell

Szeliski [32] munkája alapján a pinhole (lyukkamera) modell a 3D-s tér és a 2D-s kép közötti leképzés legegyszerűbb geometriai leírása, amelyben a fénysugarak egyetlen ponton, a vetítési középponton haladnak át, lencsék alkalmazása nélkül. A modell matematikai alapját a hasonló háromszögek képezik, amelyek segítségével a rendszer egy 3D-s pont (x, y, z) koordinátáit a képsík (x, y) koordinátáira vetíti, ahol az összefüggést a fókusztávolság (f) és a mélység (z) aránya határozza meg: "$x = f \times \frac{x}{z}$" és „$y = f \times \frac{y}{z}$". A könyv részletezi, hogy a számítógépes látásban ezt a leképzést gyakran homogén koordinátákkal és egy 3×4-es vetítési mátrixszal írják le, amely szétválasztja a kamera belső paramétereit (mint a fókusztávolság és a képközéppont eltolása) és a külső paramétereket (a kamera térbeli pozícióját és orientációját). Bár a modell idealizált és nem számol a lencsetorzításokkal vagy az elmosódással, Szeliski kiemeli, hogy ez szolgál alapul a bonyolultabb képalkotási rendszerek megértéséhez és a kamerakalibrációs eljárásokhoz is.

Inverz vetítés

Szeliski könyve alapján a pixel-koordináták világ koordinátákká való transzformációja, azaz az inverse projection (inverz vetítés) egy eredendően többértelmű probléma, mivel a 3D tér 2D síkra történő leképezése során a mélységi információ elvész. A szerző részletesen kifejti, hogy a lyukkamera modell matematikai keretrendszerében egy adott képpont visszavetítése, nem egyetlen konkrét térbeli pontot, hanem a kamera vetítési középpontjából kiinduló, és a képponton áthaladó ray sugarat határoz meg, a háromdimenziós térben. Ebből adódóan a pontos világkoordináták (x, y, z) helyreállításához a kamera belső és külső paramétereit tartalmazó vetítési mátrix inverzének alkalmazása önmagában nem elegendő. A megoldáshoz ahogy Szeliski a geometriai képalkotásról szóló fejezetekben tárgyalja elengedhetetlen valamilyen kiegészítő geometriai felvetés, információ, például ismert mélység, több nézőpontból származó háromszögelés vagy egy ismert síkkal, talajjal való metszéspont meghatározása [32].

Kamera kalibráció

Richard Szeliski könyve alapján ismertetem a geometriai kamera kalibráció folyamatát, amely a számítógépes látás egyik legalapvetőbb lépése a 3D-s tér rekonstrukciójához

[32].

A geometriai kamera kalibráció az a folyamat, melynek során meghatározzuk a kamera belső geometriai és optikai tulajdonságait, valamint a térbeli helyzetét. A kalibráció elengedhetetlen minden olyan alkalmazásnál, ahol a 2D-s képi információból pontos metrikus következtetéseket kívánunk levonni a 3D-s valóságról. Nyers, kalibrálatlan képek esetén ugyanis a lencsetorzítás és a vetítési paraméterek ismeretének hiánya, lehetetlenné teszi a precíz távolságbecslést vagy a képen szereplő tárgyak méretének meghatározását.

A kalibrációs modellt a lyukkamera modellre építi, amelyet a további lépésekkel egészít ki. A szerző két lépést különít el a kalibrációs folyamat során, belső paraméterek és külső paraméterek.

A belső paraméterek a kamera belső tulajdonságait írják le, amelyek állandóak tekinthetőek. Ide tartozik a fókusztávolság, amely a nagyítás mértékét határozza meg, valamint az optikai középpont, ahol az optikai tengely metszi a képsíkot. Továbbá ki hangsúlyozza, hogy a valós lencsék, különösen a nagylátószögűek, radiális torzítást okoznak (a képek széle görbül), amivel a kalibráció során számolni kell.

Külső paraméterek a kamera világkoordináta rendszerhez viszonyított helyzetét és orientációját írják le. Míg a belső paraméterek jellemzően fixek, a külső paraméterek minden egyes kép készítésnél változhatnak, ha a kamera mozog.

A gyakorlati kalibráció során ismert geometriájú kalibrációs mintákat (leggyakrabban sakktáblát) használnak. Az eljárás lépései a következők, első az adatgyűjtés, a kamera több különböző szögből készít képet a mintáról. Következő lépés a jellemző pont detektálása, az algoritmus azonosítja a minta jellegzetes pontjait (pl. a sakktábla sarkait) a képeken. Az utolsó lépésbe paraméterbecslé és optimalizáció történik, mivel a pontok valódi 3D koordinátái ismertek a mintán, és láthatóak azok 2D vetülete a képen, az algoritmus egy optimalizációs eljárással (például a legkisebb négyzetek módszerével vagy a Levenberg-Marquardt algoritmussal), addig finomítja a kamera paramétereit, amíg a reprojekciós hiba minimális nem lesz. Ez a hiba a mért képpontok és a matematikai modell által visszavetített pontok közötti távolságot jelenti, ennek minimalizálása garantálja a modell pontosságát.

Összességében a geometriai kamera kalibráció biztosítja azt a matematikai transzformációt, amely lehetővé teszi, hogy a torzított, perspektivikus 2D képekből visszanyerhető legyen a 3D-s tér geometriailag helyes szerkezetét.

2.3.2 Geometriai transzformációk

Inverse perspective mapping

Az inverse perspective mapping (IPM, inverz perspektív leképezés) egy geometriai transzformáció, amelynek célja a kamera által rögzített képen jelentkező perspektív hatás megszüntetése. Az IPM a kamera kép származó információkat használja fel egy felülnézeti, „bird's eye view" (madártávlati) kép létrehozásához, ahol a perspektívahatások nincsenek jelen [33], [34].

Alkalmazása ADAS rendszerekben és sávészlelés Az IPM kritikus szerepet játszik az advanced driver assistance systems (ADAS, fejlett vezetéstámogató rendszerek) előfeldolgozási fázisában, különösen a sávfelismerés és sávtartás területén. A transzformáció legfőbb előnye, hogy az új koordináta rendszerben a valóságban párhuzamos útburkolati jelek (sávok) a képen is párhuzamosakká válnak, és állandó szélességűek maradnak, függetlenül a távolságtól.

A módszer gyenge pontja a síkút feltételezés. Amennyiben az út felülete nem sík például emelkedők, lejtők vagy kátyúk, egyenetlenségek esetén, a geometriai feltételezés sérül, ami a transzformált képen torzításokat eredményez [34].

A projektem kapcsán az IPM használható lehet a perspektív torzítás kiküszöbölésére. Ebből adódóan akár a sáv szelességéhez is mérhetném a detektált kátyú oldalirányú pozícióját. Persze ehhez az úton lévő sávelválasztó felfestéseket is detektálnom kellene.

Vehicle distance estimation using a mono-camera for FCW/AEB systems

Han és munkatársai tanulmánya egy monokuláris kamerát alkalmazó, távolságbecslési algoritmust mutat be az ütközésre figyelmeztető (FCW) és automatikus vészfékező (AEB) rendszerek pontosságának javítása érdekében. A szerzők a lyukkamera modellre alapozva két párhuzamos részmódszert dolgoztak ki a céljármű szélességének, és ezáltal távolságának meghatározására [35].

Tehát van egy jármű, amibe egy kamera van szerelve és veszi a környezetet beleértve a céljárművet, a céljármű az, ami a kamerás autó elött halad. Az első módszer a sávfelfestéseket detektálja és az ismert sávszélességet használja referenciaként. Míg a másik a járműre szerelt kamera magasságát és a képen lévő horizontvonalat veszi alapul olyan esetekre, amikor a sávfelismerő rendszer nem elég magabiztos.

A szerzők rámutatnak, hogy mindkét módszernek megvannak az előnyi és a hátrányai. Ugyanis amikor a kamera magassága érzékeny a pitch motion-re, ilyenkor az autó orra le fel mozog, ezért a kamera dőlésszöge változik, és elmozdul a horizont menet közben.

Pitch motion következik be hírtételen gyorsítás, nagy fékezés esetén, illetve az út egyenletlenségéből fakadóan is. Előnye, hogy nem kell ismerni a céljármű méretét, mert a járműképmagassága és horizont távolsága alapján számítja a céljármű távolságát. A szélességen alapuló módszer nem érzékeny a pitch motion-re mivel itt nem a horizonthoz viszonyítanak a szerzők, hanem magának a céljármű szélességét veszik figyelembe. Hátránya, hogy ismerni kellene a jármű valódi, fizikai szélességét, ez azonban ismeretlen. Ezért a sávszélességhez viszonyítják a céljármű szélességét. Ehhez pedig mindenképp szükséges sávfelfestés az úton, és detektálni kell azokat.

A jármű dinamikus állapotjelzőinek (pozíció, sebesség, gyorsulás) nyomon követésére a kutatásban a Kalman szűrőt alkalmazza. A szűrő minden ciklusban két fázist hajt végre predikciót és korrekciót. Predikció megjósolja, hol kellene lennie az autónak a fizikai modell alapján majd a korrekció összeveti a jóslatot a kamera által mért aktuális távolsággal és súlyozottan korrigálja az állapotot. Fontos megjegyezni, hogy a sáv nélküli ágon a kutatók egy kiterjesztett Kalman-szűrőt alkalmaznak. Erre azért van szükség, mert ott a jármű szélessége és a képi változás közötti kapcsolat nem lineáris.

A végső mérési eredményeket pedig egy megbízhatósági indexen alapuló fúziós algoritmus állítja elő a két részmódszer adatainak súlyozott egyesítésével. Ez a mechanizmus biztosítja, hogy a rendszer dinamikusan alkalmazkodjon a környezeti viszonyokhoz. Azaz, ha a sávfelfestések tiszták, azokra támaszkodik, ha bizonytalanok, akkor a horizont alapú becslésre vált.

Összességében jó rámutattak a sávészlelés fontosságát és a sávszélességhez történő viszonyítás pontosságát. Noha a szerzők nem kátyúk lokalizációját, hanem a céljármű távolságának meghatározását tárgyalják, az ismertetett eljárást kifejezetten hasznosnak ítéltem a távolságbecslés alapelveinek megértéséhez.

2.4 Nyílt forráskódú szimulációs környezetek

A fejlett vezetéstámogató rendszerek és a önvezető járművek validációja során a szimulációs környezetek alkalmazása megkerülhetetlenné vált. Ugyanis a hagyományos, nyílt közutakon zajló tesztek túlságosan drágák, sok időt emésztenek fel, és komoly balesetvédelmi, illetve etikai aggályokat vetnek fel. A virtuális tesztkörnyezetek legnagyobb előnye abban rejlik, hogy gazdaságos és a résztvevőkre nézve teljesen kockázatmentes módon teszik lehetővé a legszélsőségesebb, ritkán előforduló közlekedési forgatókönyvek korlátlan, megismételhetőségét. Fő hátrányként említhető, hogy a virtuális szenzorok zajait, a sűrű forgalom kiszámíthatatlanságát és a komplex emberi viselkedést, valamint a fizikai mozgásokat imitáló hardveres visszacsatolásokat jelenleg még nem lehet tökéletesen leképezni, ezáltal a szimulátoros vizsgálatok önmagukban sosem képesek maradéktalanul kiváltani a valódi járművekkel végzett utcai teszteket.[36]

2.4.1 CARLA

A CARLA (Car Learning to Act) az Unreal Engine 4-re épülő, nyílt forráskódú városi vezetési szimulátor, amelyet Dosovitskiy fejlesztett ki kifejezetten az önvezető rendszerek betanítására és validálására. Játékmotorként az Unreal Engine 4 fotorealisztikus renderelést és rugalmas környezetszerkesztést biztosít [37]. A szenzorkészlet kiemelkedő, RGB és mélységi kamera, LIDAR, radar, GPS, IMU, valamint szemantikai szegmentációs kamera egyaránt konfigurálható [38]. A szenzorok adatfolyamai lekérdezhetőek Python, illetve C++ API-n keresztül, teljes ROS2-integrációval [39], [40]. Hátránya, hogy „az alapértelmezett PhysX fizikai modellje nem produkál konzisztens dinamikai viselkedést" [41]. Így az út egyenetlenségből fakadó futóműreakciók és az ebből eredő kamerakép-mozgás modellezése szempontjából nem tűnik ideálisnak számomra.

2.4.2 BeamNG.tech

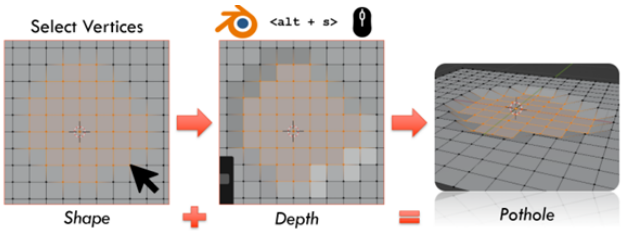
A BeamNG.tech a BeamNG GmbH által fejlesztett ADAS és önvezető jármű kutató platform, amelynek alapja egy saját fejlesztésű soft-body fizika motor, az úgynevezett mass-spring (tömeg-rugó) modellen alapul. Ez a megoldás csomópontokból és a köztük feszülő, rugalmas kötésekből álló hálózattal nagy pontossággal képes szimulálni a járművek valós idejű deformálódását, roncsolódását, a guminyomást, valamint az aerodinamikai hatásokat. Szenzorok terén elérhetőek a következők, LIDAR, IMU, ultrahang, illetve normál RGB, mélységérzékelős vagy szemantikusan szegmentált képeket adó kamerák. A rendszerrel való kommunikációt elsősorban a BeamNGpy nevű, nyílt forráskódú Python API biztosítja. A keretrendszer kiemelkedő előnye a valósághű fizikai pontosság, amellyel biztonságkritikus és extrém helyzetek (például karambolok vagy defektek) is hitelesen szimulálhatóak. [42] [43]

2.5 Szintetikus kátyú és teszt pálya generálás

2.5.1 3D modellezés

A 3D modellek felépítés három technika szerint valósítható meg, a hagyományos formális, procedurális, és a rekonstrukciós valós nyers adatokból.

A hagyományos esetben egy 3D szerkesztőprogramon (például: Blender) belül meglévő mesh grid-ek (rácshálózatok) csúcspontjainak térbeli eltolása. Például egy úton lévő kátyú esetén ahogy a 2.6. ábrán látható. A 3D modellezésekor a sík felület bizonyos csúcspontjait kijelölve egy csoportba foglalják, majd lefelé tolják a térben, kráterszerű mélyedést hozva létre. [44]



2.6. ábra: 3D kátyú modell készítés Blenderben [44]

Ezzel szemben a procedurális technikánál nem manuálisan, egy 3D-s szerkesztőprogramon keresztül történik a modellezés, hanem program kódból, matematikai algoritmusok, szabályok és vizuális csomópontok használatával. A módszer további előnye a szerkesztési idő csökkentése mellett a változatosság, randomizálhatóság azaz képes nagy mennyiségű és sokféle 3D-s tartalmat előállítani. [45]

A valós adatokból (például: kameraképekből vagy lézerszkennerekből) történő 3D rekonstrukció módszertana egy több lépcsős folyamat, amelynek felépítése során a kinyert és párosított képi jellemzőkből először egy ritka, majd a kameramozgások becslésével egy sűrű pontfelhőt generálnak, amit végül zajszűréssel és szegmentálással megtisztítva alakítanak át háromszöghálós vagy geometriai modellekké. A technológia előnye, hogy a modern algoritmusok révén nagyrészt automatizálható rendkívül részletgazdag. Ugyanakkor jelentős hátránya, hogy a terepi nyers adatok begyűjtése időigényes, drága és sokszor balesetveszélyes feladat. [46]

2.5.2 Decals (matricák)

A decal-ok (matricák) lokálisan paraméterezett, kétdimenziós textúrák, amelyeket digitális matricákként illesztenek a 3D-s objektumok felületére a részletgazdag textúrázás és a vizuális effektek dinamikus megjelenítése céljából. A matricák elsődleges szerepe a fotorealisztikus és komplex felületi részletek gyors, erőforrás kímélő kialakítása, mint amilyenek az apró részletek (például szőrzet, arcvonások) vagy a 2D-s vektoros grafikai elemek és feliratok felvitele a térbeli modellekre [47]. Modern szimulátorokban (például autonóm járművek tesztelésénél) emellett kiemelkedő szerepet kapnak a dinamikus és hiteles környezeti tényezők ábrázolásában is, ugyanis segítségükkel a pályára vetítve valósághűen szimulálhatóak az olyan időjárási viszonyok, mint az aszfalton lévő hó, eső, és a por [48].

A globális paraméterezés során a teljes 3D-s modellt vagy annak egy nagy, összefüggő darabját, egyetlen átfogó, 2D-s koordinátarendszerbe képezik le. Ezzel szemben a lokálisan paraméterezett matricák esetén a 3D-s felületnek csupán egy apró, specifikus részét képezi le egy adott középpont és egy meghatározott sugár alapján, teljesen függetlenül a

modell többi részétől. Tehát a decalok függetlenek a 3D modell egészének globális paraméterezésétől, így bármilyen felületen rá ragaszthatóak. Az elhelyezési folyamat egy úgynevezett diszkrét exponenciális leképezésen alapul. A matrica pozicionálásához csupán egy kezdőpont, egy orientációs irány és egy geodéziai sugár meghatározására van szükség. A technológia legnagyobb előnye, hogy a komplex 3D-s felületek textúrázását a 2D-s grafikai szoftverekből jól ismert egyszerűséggel teszi lehetővé, ami nagymértékben megkönnyíti a grafikai elemek mozgatását, másolását, beillesztését és többszöri újrahasznosítását egyetlen modellen belül. [47]

3 SPECIFIKÁCIÓ

3.1 Program funkciói

3.1.1 Program célja

A projektem célja, hogy egy az autóra elejére, a szélvédő mögé és a visszapillantó tükör közelében rögzített, monokuláris kamera felviteléből, miközben az autó mozog az úton, a kamera származó videofolyamot feldolgozva, a képkockákon lévő kátyúkat felismerni, detektálni (képen lévő helyüket bekeretezni).A kátyú detektálását követő lépés azok gépjárműhöz viszonyított, valós metrikus térbeli pozíciójának meghatározása. Ennek során a rendszer a kamerakép 2D-s koordinátáit valós, méterben kifejezett fizikai értékekké transzformálja, meghatározva ezzel a hibahely pontos oldalirányú távolságát. A lyukkamera modellre építve, szeretném meghatározni a kátyú oldalirányú pozícióját az úton belül, a felvételt készítő autóhoz képest. Mindezt valós időben, azaz miközben a jármű előrefelé halad az úton a szoftvernek az autó előtti területet kell vizsgálnia, és előre detektálni a kátyúkat és azok oldalirányú pozícióit, mielőtt az autó oda érne a kátyús útszakasz részhez. Tehát egyfajta szoftveres előre tekintés az úton, ami nem a lámpákat, közlekedési táblákat és nem a felfestéseket figyeli az útburkolaton, hanem az úthibákat, azon belül is kifejezetten a kátyúkra fókuszál.

Meglátásom szerint egy ilyen rendszerrel alapot lehetne teremteni autonóm kátyú kikerülő manőverek végrehajtására egyfajta vezetéstámogató rendszer keretein belül. Ez nemcsak a járművek állapotának megóvásában jelentene előnyt, hanem növelné a közlekedés biztonságát és nem utolsó sorban a menetkomfortot is. Ez indokolj azt a döntésemet, hogy csak egy darab monokuláris kamra képpel dolgozok. Mert a jelenlegi modern autók esetében, sok advanced driver assistance systems (ADAS) funkció egy kamera kép feldolgozásával van megoldva.

Szeretném ki hangsúlyozni, hogy a projekt keretein belül elsősorban csak a kátyúk magabiztos felismerése, azok detektálása és az úton belül betöltött oldalirányú helyzetük minél pontosabb meghatározása áll, a felvételt készítő járműhöz képest, valós időben. Tehát a rendszer nem foglalkozik a járműdinamikai beavatkozással (kormányzás), kizárólag a környezetérzékeléssel. Az esetleges vezetéstámogató rendszer rész egy ráépülő önvezetéssel kapcsolatos továbbfejlesztési ötlet, ami a motivációt adja a projekthez.

3.1.2 Elérhető funkciók részletesen

Videofolyam feldolgozás és előfeldolgozás, a rendszer belépési pontja a valós idejű képfeldolgozó modul, amely a szélvédő mögött elhelyezett monokuláris kamera jelét fo-

gadja.

Első lépésben a szoftver folyamatosan olvassa és digitalizálja a kamera által biztosított 2D videofolyamot. A feldolgozási lánc biztosítja, hogy a rendszer képes legyen tartani a bemeneti forrás sebességét (szinkronizált 30 FPS), ezáltal garantálva a folyamatos, akadozásmentes működést nagy sebességű haladás esetén is.

Majd mielőtt a detektáló algoritmus lefutna, a rendszer előfeldolgozást végez az egyes képkockákon, hogy a nappali, természetes fényviszonyok mellett a lehető legkontrasztosabb képet biztosítsa a kátyúk körvonalainak kiemeléséhez.

Kátyú detektálás, objektum felismerés modul a rendszer szeme, amely a vizuális információk értelmezéséért felelős, gépi tanulás segítségével.

A szoftver képes azonosítani az aszfaltozott útfelületen felszínen található durva, egyenletlen tál alakú mélyedés, azaz kátyúkat. A funkció kritikus eleme a megkülönböztetés képesség, tehát az algoritmus robusztusan különíti el a kátyúkat az egyéb vizuális zajoktól. A rendszer nem tekinti kátyúnak az árnyékokat, az útburkolati jeleket, az aszfalt foltszerű javításait vagy a repedéseket, így minimalizálva a false positive-ok számát.

A funkció előre tekintő detektálást végez, a jármű előtti teret pásztázva, már 30 méteres távolságból azonosítsa a potenciális veszélyforrásokat. Ez az előre tekintés biztosítja a szükséges reakcióidőt a sofőr vagy egy későbbi autonóm rendszer számára.

Térbeli Lokalizáció és Koordináta-transzformáció a rendszer legfontosabb matematikai modulja, amely a detektált objektumokat elhelyezi a valós térben a járműhöz viszonyítva.

2D-ből 3D transzformációhoz a lyukkamera-modell matematikai összefüggéseit alkalmazva a szoftver a sík képi koordinátákat átkonvertálja a 3D világkoordináta rendszerbe. Ez a számítás teszi lehetővé, hogy a képernyőn látott foltból valós fizikai távolságadat legyen.

Laterális (oldalirányú) pozíció meghatározásánál a rendszer kiszámítja a detektált kátyú oldalirányú elhelyezkedését a jármű hossztengelyéhez (orrához) képest. A funkció célja, hogy centiméteres pontossággal megadja, az autó nyomtávjához viszonyítva hol helyezkedik el a kátyú az úton.

3.2 Követelmények

3.2.1 Követelmények kifejtve

Funkcionális követelmények:

Ezen követelmények határozzák meg a rendszer alapvető működési mechanizmusait, a bemeneti adatok kezelésétől kezdve a feldolgozáson át egészen az eredmények megjelenítéséig.

Bemeneti videófolyam feldolgozása A rendszer elsődleges bemeneti interfészének képesnek kell lennie a jármű szélvédője mögé, a visszapillantó tükör környezetében rögzített monokuláris kamera jelének fogadására. A szoftvernek folyamatosan, megszakítás nélkül kell feldolgoznia a kamera által biztosított, kétdimenziós videófolyamot. Ez a funkció képezi a környezetérzékelés alapját, így elengedhetetlen, hogy a rendszer a képkockák egymásutániságát időben helyesen kezelje, biztosítva ezzel a későbbi modulok számára szükséges adatfolyamot.

Szelektív detektálás és objektumosztályozás A képfeldolgozó algoritmusnak magas megbízhatósággal kell azonosítania az útfelületen található geometriai mélyedéseket, azaz a kátyúkat. A funkció kritikus eleme a szelektivitás, a rendszernek képesnek kell lennie megkülönböztetni a valódi veszélyt jelentő úthibákat az egyéb vizuális zajoktól. A detektálási logikának ki kell szűrnie a false positive-okat, amelyeket az útburkolati jelek, az aszfalt repedései, a foltszerű javítások, vagy a környezeti tárgyak (fák, épületek) által vetett árnyékok okozhatnak. Kizárólag a jármű futóművére veszélyes mélyedések kerülhetnek detektálásra.

Valós idejű vizualizáció és visszajelzés, a fejlesztés, tesztelés és a későbbi felhasználói felügyelet támogatása érdekében a rendszernek vizuális visszajelzést kell biztosítania a feldolgozás eredményéről. A feldolgozást végző eszköz (laptop) képernyőén megjelenő élő videóképen a detektált kátyúkat egyértelműen, a kiterjedésüket jelölő határoló dobozzal (bounding box) kell kiemelni. Ezen felül a vizualizációnak tartalmaznia kell a számított numerikus adatokat is, különös tekintettel a kátyú járműtől mért távolságára és az oldalirányú pozíciójára, ezzel segítve a rendszer pontosságának szubjektív és objektív ellenőrzését.

Működési tartomány a rendszer alkalmazhatósága a projekt jelenlegi fázisában meghatározott útviszonyokra és járműmozgásra korlátozódik. A detektálási algoritmusnak aszfaltozott burkolattal ellátott utakon kell üzembiztosan működnie. A geometria számítások egyszerűsítése és a lyukkamera-modell érvényessége érdekében a rendszernek elsősorban egyenes vonalú haladáskor kell helytállnia. A specifikáció kizárja a rendszer működéséből a tolatási manővereket, valamint a hirtelen irányváltással járó éles kanyarokat, mivel ezekben az esetekben a kamerakép és a járműmozgás vektora közötti korreláció jelentősen eltér a modell feltételezéseitől.

Minőségi követelmények:

A minőségi követelmények definiálják azokat a teljesítménymutatókat és korlátokat,

amelyeken belül a funkcionális követelményeknek teljesülniük kell a biztonságos és hatékony működés érdekében.

Valós idejű működés, rendszer reakcióképességének kulcsa, hogy a teljes feldolgozási lánc, beleértve a kép előfeldolgozását, a detektálást, és a matematikai lokalizációt, képes legyen követni a bemeneti videó sebességét. Mivel a kamera 30 képkocka/másodperc (FPS) sebességgel rögzít, a szoftvernek rendelkeznie kell azzal a számítási kapacitással, hogy egyetlen képkocka feldolgozása ne haladja meg a ~33 milliszekundumot. Ez biztosítja, hogy ne alakuljon ki késleltetés, ami a gyorsan mozgó jármű esetén adatvesztéshez és(vagy) elkésett detektáláshoz vezetne.

Detektálási távolság és előre tekintés, a szoftvernek képesnek kell lennie arra, hogy a kátyúkat 30 méteres távolságból azonosítsa. Ez a távolság elegendő időt biztosítana egy elméleti autonóm rendszernek vagy a sofőrnek a kikerülő manőver megtervezésére és végrehajtására, még nagyobb sebesség mellett is.

Támogatott sebességtartomány A rendszernek le kell fednie a városi és az országúti közlekedés jellemző sebességtartományait. A képfeldolgozó algoritmusoknak robusztusnak kell lenniük a mozgási elmosódással szemben, biztosítva a stabil detektálást álló helyzettől (0 km/h) egészen 70 km/h sebességig. A szoftver működése nem degradálódhat a jármű sebességének növekedésével ezen a tartományon belül.

Lokalizációs pontosság a projekt egyik legfontosabb műszaki kihívása a 2D képpontok 3D térbeli koordinátákká való konvertálása. A járműhöz közel, 10 méteres távolságon belül a kátyú oldalirányú pozíciójának becslési hibája nem haladhatja meg a ±15 cm-t. Ez a pontosság kritikus fontosságú, mivel egy átlagos gumiabroncs szélessége is összemérhető ezzel az értékkel.

A minél jobb környezeti robusztussághoz rengeteg adatra és számítási kapacitásra van szükség. Ezért a szoftvernek csak nappali, természetes fényviszonyok mellett kell megbízhatóan üzemelnie. Ez magában foglalja a napsütéses, felhős vagy változóan árnyékos időszakokat is, azonban a jelenlegi fázisban nem terjed ki az éjszakai vezetésre vagy az extrém időjárási körülmények (köd, szakadó eső, havas táj) kezelésére.

3.2.2 Követelmények felsorolás jelleggel

Funkcionális követelmények:

1. Bemenet: A rendszernek fel kell dolgoznia a szabványos, monokuláris kamera által biztosított 2D videófolyamot.
2. Szelektív detektálás: A szoftvernek képesnek kell lennie a kátyúk megkülönböztetésére egyéb útburkolati jelektől (felfestések, repedések, foltok, árnyékok).

3. Vizualizáció: A detektált hibákat a képernyőn határoló dobozzal és a számított távolságadatok megjelenítésével kell jelölni.
4. Működési tartomány: A detektálásnak aszfaltozott egyenes úton, egyenes vonalú haladás esetén kell működnie, tolatás és éles kanyarok esetén nem.

Minőségi követelmények:

1. Valós idejűség: A teljes feldolgozási láncnak (pre-processing, detektálás, lokalizáció) követnie kell a bemeneti 30FPS videó sebességét.
2. Detektálási távolság: A rendszernek legalább 30 méter távolságból fel kell ismernie a kátyúkat.
3. Sebességtartomány: A funkciónak 0–70 km/h járműsebesség mellett stabilan kell működnie.
4. Lokalizációs pontosság: A kátyú oldalirányú pozíciójának becslési hibája a járműtől 10 méteren belül nem haladhatja meg a ±15 cm-t.
5. Robusztusság: A rendszernek nappali, természetes fényviszonyok mellett kell üzembiztosan működnie.

3.3 Megvalósítási módszertan

A rendszer komplexitása indokolta a hibrid fejlesztési módszertan alkalmazását, amely ötvözi a modern, adat vezérelt mesterséges intelligenciát a klasszikus, determinisztikus matematikai modellezéssel. A fejlesztés során két eltérő, de egymást kiegészítő stratégiát használok

3.3.1 Környezetérzékelés: Iteratív Kísérleti Fejlesztés

A kátyúk vizuális felismeréséért felelős modul létrehozása iteratív (ciklikus) fejlesztési modell alapján történt, amely a gépi tanuláson alapuló projektekre jellemző. A folyamat az adatgyűjtés, annotálás, modelltanítás és kiértékelés (validáció) ismétlődő köreiből állt. A teszteredmények folyamatos visszacsatolása alapján finomhangolm a modell hiperparamétereit.

3.3.2 Lokalizáció: Analitikus Modellalapú Rekonstrukció

A detektált hibák laterális pozíciójának és távolságának meghatározásához deduktív, matematikai modellalapú módszertant alkalmazok. Ez a megközelítés nem tanuláson, hanem szigorú geometriai szabályokon, konkrétan a lyukkamera modell alkalmazásán alapul. A módszer a síkút feltételezésre építve. A számítások pontosságát a kamera belső és külső paramétereinek előzetes kalibrációja biztosítja.

4 RÉSZLETES RENDSZERTERV

4.1 Rendszer architektúra

A szoftver belső működése egy szigorúan szekvenciális pipeline-ra épül. Az alábbiakban a rendszer szoftveres komponenseinek technikai kapcsolatait és az adatok transzformációját ismertetem.

4.1.1 Adatfolyam

1. Nyers adatbeolvasás: A folyamat a kamera által rögzített analóg optikai jel digitalizálásával kezdődik. A bemenet egy W×H felbontású, 3 csatornás (RGB/BGR) mátrix.
2. Geometriai korrekció: A nyers képkockán a rendszer lencsetorzítás mentesítést végez az előzetes kalibrációs paraméterek alapján. Az adat típusa itt még változatlanul kép.
3. Következtetés: A korrigált kép a detektálás modulhoz kerül. Ezen a ponton az adat jellege megváltozik: a kimenet már nem kép, hanem egy strukturált objektumlista, amely tartalmazza a detektált kátyúk osztályát, konfidencia értékét és a határoló dobozának koordinátáit.
4. Transzformáció: A 2D pixelkoordináták a geometriai modulba kerülnek, amely előállítja a valós térbeli (X, Y) koordinátákat (méterben).
5. Megjelenítés: A metaadatok (dobozok és távolságok) visszaírásra kerülnek az eredeti videofolyamra, létrehozva a felhasználó számára látható kimenetet.

4.2 Fejlesztői környezet

4.2.1 Fejlesztői munkaállomás

A szoftver fejlesztése, a tanító adathalmazok kezelése és a mélytanulási modell betanítása, finomhangolása egy dedikált asztali konfiguráción történik. A gép specifikációjának kiválasztásakor a legfontosabb szempont a GPU memóriamérete (VRAM) és a CUDA magok száma volt a párhuzamosított számítások gyorsítása érdekében.

hardver specifikációja:

- CPU: Intel Core i5-6600 (4 mag, 3.30 GHz), felelős az operációs rendszer futtatásáért és az adatok előkészítéséért (data loading) a GPU számára.
- RAM: 16GB DDR4, ez a memóriaigény biztosítja a nagyméretű képállományok és a tanítási batch-ek akadálymentes betöltését.

– GPU: NVIDIA GeForce GTX 1060 6GB, a konfiguráció kulcseleme. A 6GB dedikált videomemória lehetővé teszi a közepes komplexitású konvolúciós hálózatok hatékony tanítását és a valós idejű inferencia tesztelését magas képfrissítési ráta mellett.

4.2.2 Szimulációs környezet

A szakirodalomban elterjedt platformok, különösen a CARLA és a BeamNG.tech, összehasonlítása alapján a projekt megvalósításához az utóbbi környezet alkalmazása indokolt számomra. A döntést a vizuális és dinamikai valósághűség, a validálhatóság.

A monokuláris kamerakép alapján történő 2D-3D geometriai képtranszformáció egyik legkritikusabb paramétere a kamera pozíciója és orientációja az útfelülethez képest. Valós körülmények között az út egyenetlenségei, valamint a jármű mozgásállapot változásai folyamatosan módosítják a karosszéria dőlésszögét. Míg a CARLA alapértelmezett motorja hajlamos túlságosan stabil, a fizikai zajoktól mentes kameraképpel operálni, addig a BeamNG.tech saját fejlesztésű, soft-body fizikája kiemelkedő pontossággal szimulálja a jármű viselkedését a feladatom szempontjából.

4.2.3 Fejlesztői környezet, programozási nyelv, könyvtárak és eszközök

A rendszerfejlesztés elsődleges programozási nyelve a Python. Ez a nyelv a jelenlegi iparági szabvány a mesterséges intelligencia és a gépi látás területén. Mégis a fő indok a Python mellett az, hogy a BeamNG.tech szimulátor hivatalos kommunikációs interfésze (BeamNGpy) is Python alapú, ami, így garantálja a komponensek zökkenőmentes integrációját a szimulátorba. Továbbá a projekt kulcstechnológiái (PyTorch, OpenCV, Ultralytics YOLO) natív Python támogatással rendelkeznek.

Alkalmazott könyvtárak és keretrendszerek:

BeamNGpy: A rendszer egyik legfontosabb specifikus könyvtára. Ez a nyílt forráskódú Python API biztosítja a kétirányú kommunikációt a BeamNG.tech szimulátorral. Ezen keresztül történik a virtuális környezet inicializálása, a vizsgált jármű (ego-vehicle) monokuláris kamerájának paraméterezése, a képi adatfolyamok kinyerése, valamint a validáláshoz szükséges fizikai referenciaadatok (ground truth metrikus pozíciók, járműorientáció) valós idejű lekérdezése.

OpenCV (Open Source Computer Vision Library): A képfeldolgozási pipeline váza. Nem csupán egy hagyományos videofolyam, hanem a BeamNGpy által másodpercenként szolgáltatott egyedi képkockák beolvasásáért és előfeldolgozásáért felel. Továbbá ez a könyvtár biztosítja a laterális távolságméréshez szükséges geometriai és perspektív

transzformációk végrehajtását, valamint a detektált kátyúk és a számított metrikus adatok képernyőn történő valós idejű vizuális megjelenítését.

NumPy (Numerical Python): A rendszer matematikai motorja. A memóriafolytonos tárolást alkalmazó NumPy tömbök (ndarray) nagyságrendekkel gyorsabb adatkezelést tesznek lehetővé. A projektben a kamerából származó pixeladatok mátrixként való kezelésén túl a BeamNG.tech-ből érkező téri adatok feldolgozására, valamint a 2D-3D lokalizációhoz elengedhetetlen lineáris algebrai műveletek (pl. homográfia mátrixszal való szorzás, vektorizált koordináta-transzformációk) gyors elvégzésére szolgál.

Ultralytics YOLO (v10) és PyTorch: A kátyúdetektálást végző mesterséges intelligencia modul magja. A PyTorch egy alacsony szintű mélytanulási keretrendszer, amely dinamikus számítási gráfokat használ, míg az Ultralytics könyvtár egy magas szintű absztrakciót biztosít a YOLO (You Only Look Once) architektúra tanításához, finomhangolásához és futtatásához. Ez a modul felel azért, hogy a szimulált képeken a kátyúk 2D-s képkoordinátáit a képtranszformációs fázis számára azonosítsa.

Fejlesztői eszközök:

- IDE: Visual Studio Code (könnyűsúlyú kódolás és a Python környezetek egyszerű integrációja).
- Verziókezelés: Git (a forráskód változásainak követésére).
- Csomagkezelés: Pip / Anaconda (a függőségek és virtuális környezetek izolált kezelésére).

4.3 Modulok felépítése

4.3.1 Adatmodell és struktúrák

Kátyú objektum főbb attribútumai A rendszer központi adategysége a Kátyú objektum. Amikor a detektor felismer egy úthibát, példányosít egy ilyen objektumot, amely a feldolgozási lánc végéig kíséri az adott hibát, folyamatosan bővülve új attribútumokkal.

- Azonosító (ID): Egyedi egész szám, amely lehetővé teszi a kátyú követését több képkockán keresztül.
- Határoló doboz (bounding box): A detektált kátyú bounding box koordinátái a képen (x, y, w, h) formátumban.
- Megbízhatóság (confidence): A mélytanuláson alapuló algoritmus által visszaadott valószínűségi 0,0 – 1,0 közötti érték, amely jelzi, mennyire biztos a rendszer a detektálás helyességében.
- Valóspozíció (real world coordinates): A lokalizációs modul által számított 3D pozícióvektor X_lat, Y_long, amely méterben vagy centiméterben adja meg a hiba

távolságát a jármű referenciapontjához képest.

Kamera objektum főbb attribútumai Ez az osztály felelős az autóra rögzített kamera elhelyezéséének pontos paramétereiért.

- Magassága (H_cam): 1,45 méter a földtől.
- Oldalirányú eltolása (T_x): a jármű középvonalától 0,0 méterre.
- Hosszirányú eltolása (T_y): a lökhárítótól való távolság 0,85 méterre.
- A kamera bólintási szöge (0_pitch): 0,05236 radiánban (3fok).

4.3.2 Fő vezérlési ciklus

A rendszer működését egy központi vezérlőmodul koordinálja. Ennek a modulnak a feladata a rendszerindítás, az erőforrások menedzselése, a feldolgozási lánc (pipeline) elemeinek szinkronizált meghívása, valamint a futás felügyelete. Programozástechnikai szempontból ez a modul tartalmazza a belépési pontot (main függvény) és a végtelen ciklust (while loop).

Inicializálás és erőforrás foglalás A program indításakor a vezérlő első feladata a szükséges statikus erőforrások betöltése a memóriába, még a feldolgozási ciklus megkezdése előtt. Ekkor történik a konfiguráció betöltése. Mint a rendszer paramétereinek (pl. kamerakalibrációs mátrixok, detektálási küszöbértékek, ROI beállítások) beolvasása külső fájlokból. Modell inicializálás: A YOLO hálózat súlyainak betöltése a GPU memóriájába, hogy az első képkocka feldolgozása már ne szenvedjen késedelmet. Kamerakapcsolat létesítés a cv2.VideoCapture interfész megnyitása és a kamera paramétereinek (felbontás, FPS) ellenőrzése.

A valós idejű feldolgozási ciklus A rendszer egy végtelen ciklus működteti, amely addig fut, amíg a felhasználó leállítási parancsot nem ad. A cikluson belül szigorú szekvenciális végrehajtás történik, ezzel biztosítva az adatok konzisztenciáját.

A vezérlő lekéri a legfrissebb nyers képkockát a kamera pufferéből. Amennyiben az olvasás sikertelen, a modul hibaüzenetet generál vagy megkísérli az újra csatlakozást.

Sikeres lekérés után a modulhívások láncolata következik. A vezérlő sorrendben átadja az adatokat az alrendszereknek:

1. Előfeldolgozó: a nyers képből torzításmentesített képet készít.
2. Detektor: A bementi képből elkészíti az objektum listát.
3. Lokalizáló: Elvégzi konverziót és kinyeri a távolság adatokat.

4. Megjelenítő: A feldolgozás során kinyert adatokat rá illeszti a videó képre és megjeleníti azt.

Teljesítmény monitorozás: A vezérlő minden ciklusban méri a futási időt, és ebből valós időben számítja az FPS értéket. Ez az adat visszacsatolást ad a rendszer terheltségéről.

A szoftver futásának ellenőrzött, manuális megszakítása a felhasználó által kezdeményezhető az erre a célra dedikált ESC billentyű lenyomásával. A vezérlő modul a fő feldolgozási ciklus minden iterációjában figyeli a billentyűzet-eseményeket, és a megszakítási kérelem észlelésekor azonnal kilép a végrehajtási hurokból, elindítva a szabályos leállítási mechanizmust.

4.3.3 Előfeldolgozás

Képarány helyes átméretezését a korrigált képet a YOLO által megkövetelt fix felbontásra 640x640 pixel méretezi át (cv2.resize). Az eredeti képarány megtartása mellett skálázza a képet, a fennmaradó üres területeket (cv2.copyMakeBorder) pedig kitöltő színnel (padding) látja el. Ezzel elkerülhető a detektált objektumok alakjának torzulása.

Majd a normalizálás és formázás transzformációs lépések a PyTorch keretrendszer bemeneti követelményeinek teljesítése. A színteret BGR-ről konvertálja RGB-re, a pixelértékeket a [0, 1] tartományba normalizálja, végül a mátrixot a GPU-feldolgozáshoz szükséges tenzor formátummá alakítja.

A kátyú detektálását az Ultralytics YOLOv10 architektúrája végzi. A modell kiválasztását a 2.2.5. pontban (19. oldalon) tárgyalt Optimized Road Damage Detection Challenge, nemzetközi verseny hivatalos rangsora és eredményei indokolják. A megmérettetésen az első helyezést elérő FRDC-SH csapat megoldása a YOLOv10 modellt alkalmazta, amellyel kiemelkedő egyensúlyt teremtettek a sebesség és a detektálási pontosság között.

A hálózat betanítása során alkalmazott módszertan a transfer learning-ra épül. Jelentős fejlesztési előnyt biztosít, hogy a győztes FRDC-SH csapat a GitHub platformon publikálta a előtanított modelljük súlyfájljait.

Ennek köszönhetően a tanítási folyamatot nem az alapoktól kell elkezdeni. A rendszer betölti a nagy kiterjedésű, nemzetközi adathalmazokon előzetesen betanított és optimalizált súlyokat. Ezt a folyamatot kizárólag egy célzott finomhangolási fázis követi, amelyhez a dedikált BeamNG.tech szimulációs környezetből kinyert, 2D-s virtuális kameraképeket használom fel. Ez a megközelítés garantálja, hogy a modell a szimulátor specifikus vizuális és dinamikai sajátosságaira optimalizálódjon, miközben az előtanított súlyok révén drasztikusan csökken a szükséges tanítási idő és a hardveres erőforrásigény.

4.3.4 Kátyú lokalizálás

Számítás menete A számítás megkezdése elött megvizsgálja a kátyú osztály példánynak a megbízhatóság (confidence) értékét, ha az nagyobb mint 0,5 akkor belekezd a számításba.

A Kátyú objektumból (Bemenet):

- x, y, w, h: A határoló doboz paraméterei pixelben.

A Kamera objektumból (Konstansok):

- H_cam: Kamera magassága (méterben, pl. 1.45).
- T_x: Kamera oldalirányú eltolása a jármű középpontjától (méterben).
- T_z: Kamera hosszirányú eltolása a jármű referenciapontjától (pl. lökhárítótól) (méterben).
- 0_pitch: A kamera bólintási szöge

Kamera belső paraméterek:

- f_x, f_y: Fókusztávolság pixelben.
- c_x, c_y: Az optikai középpont koordinátái pixelben.

1. Lépés a talppontot meghatározása. A kátyú térbeli helyzetét annak az útfelülethez legközelebb eső pontja határozza meg. A bounding box alapján a referenciapont (u_ref, v_ref):

$$u_ref = x + w/2$$

$$v_ref = y + h$$

2. Lépés: a beesési szög számítása. A képpont függőleges koordinátájából (v_ref) meghatározza a fénysugár beesési szögét az optikai tengelyhez képest

$$alpha = arctan((v_ref - c_y)/f_y)$$

A teljes szöget (gamma) az útfelület normálisához (függőlegeshez) képest a kamera fizikai dőlésszöge (0_pitch) és a pixeltől függő szög összege adja:

$$gamma = 0_pitch + alpha$$

3. Lépés: a hosszirányú távolság számítása (Kamera koordinátákban). A síkút felté-

telezés (Z_worl = 0) alapján egy derékszögű háromszög alakul ki, ahol az egyik befogó a kamera magassága (H_cam), a másik pedig a keresett távolság (d_optical) a talajon.

$$d_optical \ = \ H_cam \ / \ tan(gamma)$$

Ez az érték a kamera vetületétől mért távolság az úton.

4. Lépés: az oldalirányú pozíció számítása (Kamera koordinátákban). A vízszintes pixelkoordináta (u_ref) és a fókusztávolság (f_x) aránya megegyezik a valós oldalirányú kitérés (x_cam) és a mért távolság (d_optical) arányával:

$$x_cam \ = \ (u_ref \ - \ c_x) \ * \ (d_optical \ / \ f_x)$$

5. Lépés: transzformáció a Jármű-koordinátarendszerbe. Utolsó lépésként a kamera koordináta rendszerében kapott értékeket átültetjük a jármű rögzített koordinátáiba. Mivel a kamera a szélvédő mögött van a hosszirányú távolságból ezt az értéket le kell vonni. Hosszirányú pozíció (Y_long):

$$Y_long \ = \ d_optical \ - \ T_z$$

(Ha Y_éong negatív, a kátyú már a lökhárító vonala mögött van).

Oldalirányú pozíció (X_lat):

$$X_lat \ = \ x_cam \ + \ T_x$$

Bólintási zaj csökkentése A számításra legkritikusabb hibaforrása a jármű felfüggesztésének mozgásából eredő dinamikus bólintó hatás, ami által az állandónak tekintett pitch (0_pitch) nem lesz igaz. Így a kátyú számított hosszirányú pozíciójában jelentős, véletlenszerűnek tűnő ingadozásokat okoz. Ennek a geometriai zajnak a kiküszöbölésére a rendszer egy lineáris Kalman szűrőt alkalmazása indokolt, amely a zajos méréseket egy determinisztikus fizikai mozgásmodellel ötvözi.

A szűrő működésének alapelve azon a fizikai tényen nyugszik, hogy az úthiba statikus objektum, a jármű pedig tehetetlenségéből adódóan rövid időintervallumon belül egyenletes sebességgel közeledik felé, így a kátyú valós távolságának változása nem lehet ugrásszerű.

4.3.5 Megjelenítés

Grafikus annotáció (Bounding Box kirajzolás) A modul bemenetét a detektált és szűrt kátyú objektumok listája képezi. A rendszer iterál ezen a listán, és minden egyes objektumhoz tartozó pixelkoordináták (x, y, w, h) alapján egy határoló keretet (bounding box) rajzol a videóképre. A megvalósításhoz az OpenCV könyvtár rectangle függvényét alkalmazza a szoftver. A keret színe és vastagsága konfigurálható, biztosítva a láthatóságot változó fényviszonyok mellett is.

Metrikus adatok kijelzése A puszta detektáláson túl a rendszer legfontosabb kimenete a távolságbecslés. A modul a boounding box közvetlen közelében, szöveges formátumban megjeleníti a lokalizációs alrendszer által számított valós fizikai távolságokat. A szoftver a putText eljárás segítségével írja ki a jármű orrától mért longitudinális (Y_long) és a középvonaltól mért laterális (X_lat) távolságot méterben vagy centiméterben.

Rendszerállapot és teljesítmény-monitorozás A fejlesztés és a valós idejű tesztelés során kritikus fontosságú a hardveres erőforrások kihasználtságának nyomon követése. A modul a képernyő sarkában folyamatosan frissülő információs sávot jelenít meg. Itt kerül kiírásra a vezérlő modul által mért pillanatnyi képfrissítési sebesség (FPS). Ez az adat szolgál elsődleges validációként arra, hogy a rendszer képes-e tartani a specifikációban előírt valós idejű működést a feldolgozás során.

5 IRODALOMJEGYZÉK

[1] A. Abed, M. Rahman, N. Thom, D. Hargreaves, L. Li és G. Airey, „Analysis and Prediction of Pothole Formation Rate Using Spatial Density Measurements and Pavement Condition Indicators”, *Transportation Research Record: Journal of the Transportation Research Board*, 2677. évf., 11. sz., 651–664. old., 2023. nov., ISSN: 0361-1981, 2169-4052. DOI: 10.1177/03611981231166684

[2] J. S. Miller és W. Y. Bellinger, „Distress Identification Manual for the Long-Term Pavement Performance Program”, Office of Infrastructure Research and Development, McLean, FHWA-HRT-13-092, 2014.

[3] Y. Safyari, M. Mahdianpari és H. Shiri, „A Review of Vision-Based Pothole Detection Methods Using Computer Vision and Machine Learning”, *Sensors*, 24. évf., 17. sz., 5652. old., 2024. aug. 30., ISSN: 1424-8220. DOI: 10.3390/s24175652

[4] L. Yalew, G. Virgianto, M. Inagi és K. Tomiyama, „IMPACT OF A POTHOLE ON ROAD USER RESPONSE IN TERMS OF DRIVING SAFETY AND COMFORT FOR PAVEMENT MAINTENANCE PRIORITIZATION”, *Journal of JSCE*, 11. évf., 2. sz., n/a., 2023, ISSN: 2187-5103. DOI: 10.2208/journalofjsce.23-21035

[5] E. A. Martinez-Ríos, M. R. Bustamante-Bello és L. A. Arce-Sáenz, „A Review of Road Surface Anomaly Detection and Classification Systems Based on Vibration-Based Techniques”, *Applied Sciences*, 12. évf., 19. sz., 9413. old., 2022. szept. 20., ISSN: 2076-3417. DOI: 10.3390/app12199413

[6] C. Koch és I. Brilakis, „Pothole detection in asphalt pavement images”, *Advanced Engineering Informatics*, 25. évf., 3. sz., 507–515. old., 2011. aug., ISSN: 14740346. DOI: 10.1016/j.aei.2011.01.002

[7] E. M. I. R. BUZA, S. A. M. I. R. OMANOVIC és A. L. V. I. N. HUSEINOVIC, „Pothole Detection with Image Processing and Spectral Clustering”, *Recent Advances in Computer Science and Networking*, 810. köt., San Diego: WSEAS Press, 2013, 48–53. old., ISBN: 978-960-474-336-0.

[8] Y. O. Ouma és M. Hahn, „Pothole detection on asphalt pavements from 2D-colour pothole images using fuzzy c -means clustering and morphological reconstruction”, *Automation in Construction*, 83. évf., 196–211. old., 2017. nov., ISSN: 09265805. DOI: 10.1016/j.autcon.2017.08.017

[9] T. D. Chung és M. K. A. A. Khan, „Watershed-Based Real-time Image Processing for Multi-Potholes Detection on Asphalt Road”, *2019 IEEE 9th International Conference on System Engineering and Technology (ICSET)*, Shah Alam, Malay-

sia: IEEE, 2019. okt., 268–272. old., ISBN: 978-1-7281-0758-5. DOI: 10.1109/ICSEngT.2019.8906371

[10] T. Y. Goh, S. N. Basah, H. Yazid, M. J. Aziz Safar és F. S. Ahmad Saad, „Performance analysis of image thresholding: Otsu technique", *Measurement*, 114. évf., 298–307. old., 2018. jan., ISSN: 02632241. DOI: 10.1016/j.measurement.2017.09.052

[11] R. R és S. S, „A Comprehensive Review on Asphalt Pavement Distress Detection and Assessment Based on Artificial Intelligence", *2022 IEEE 9th Uttar Pradesh Section International Conference on Electrical, Electronics and Computer Engineering (UPCON)*, Prayagraj, India: IEEE, 2022. dec. 2., 1–6. old., ISBN: 979-8-3503-3250-6. DOI: 10.1109/UPCON56432.2022.9986460

[12] N. Ma és tsai., „Computer vision for road imaging and pothole detection: A state-of-the-art review of systems and algorithms", *Transportation Safety and Environment*, 4. évf., 4. sz., tdac026., 2022. nov. 21., ISSN: 2631-4428. DOI: 10.1093/tse/tdac026

[13] S. Ren, K. He, R. Girshick és J. Sun, „Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39. évf., 6. sz., 1137–1149. old., 2017. jún. 1., ISSN: 0162-8828, 2160-9292, 1939-3539. DOI: 10.1109/TPAMI.2016.2577031

[14] J. Redmon, S. Divvala, R. Girshick és A. Farhadi, „You Only Look Once: Unified, Real-Time Object Detection", *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA: IEEE, 2016. jún., 779–788. old., ISBN: 978-1-4673-8851-1. DOI: 10.1109/CVPR.2016.91

[15] J. Terven, D.-M. Córdova-Esparza és J.-A. Romero-González, „A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS", *Machine Learning and Knowledge Extraction*, 5. évf., 4. sz., 1680–1716. old., 2023. nov. 20., ISSN: 2504-4990. DOI: 10.3390/make5040083

[16] M. Hussain, „YOLO-v1 to YOLO-v8, the Rise of YOLO and Its Complementary Nature toward Digital Manufacturing and Industrial Defect Detection", *Machines*, 11. évf., 7. sz., 677. old., 2023. jún. 23., ISSN: 2075-1702. DOI: 10.3390/machines11070677

[17] L. Liu és tsai. „Deep Learning for Generic Object Detection: A Survey", elérés dátuma 2025. nov. 14. cím: http://arxiv.org/abs/1809.02165 preprint.

[18] K. Vinoth és S. P, „Lightweight object detection in low light: Pixel-wise depth refinement and TensorRT optimization", *Results in Engineering*, 23. évf., 102510. old., 2024. szept., ISSN: 25901230. DOI: 10.1016/j.rineng.2024.102510

[19] M. Hussain. „YOLOv5, YOLOv8 and YOLOv10: The Go-To Detectors for Real-time Vision”. 1. verzió, elérés dátuma 2025. nov. 16. cím: https://arxiv.org/abs/2407.02988 preprint.

[20] P. Varun. „Improving INT8 Accuracy Using Quantization Aware Training and the NVIDIA TAO Toolkit”, NVIDIA Technical Blog, elérés dátuma 2025. nov. 17. cím: https://developer.nvidia.com/blog/improving-int8-accuracy-using-quantization-aware-training-and-tao-toolkit/

[21] A. Vina. „Optimizing Ultralytics YOLO models with the TensorRT integration”, Ultralytics. cím: https://www.ultralytics.com/blog/optimizing-ultralytics-yolo-models-with-the-tensorrt-integration

[22] NVIDIA. „NVIDIA A100 Tensor Core GPU”, NVIDIA. cím: https://www.nvidia.com/en-us/data-center/a100/

[23] A. Wang és tsai. „YOLOv10: Real-Time End-to-End Object Detection”, elérés dátuma 2025. nov. 17. cím: http://arxiv.org/abs/2405.14458 preprint.

[24] F. Yang, L. Zhang, S. Yu, D. Prokhorov, X. Mei és H. Ling, „Feature Pyramid and Hierarchical Boosting Network for Pavement Crack Detection”, *IEEE Transactions on Intelligent Transportation Systems*, 21. évf., 4. sz., 1525–1535. old., 2020. ápr., ISSN: 1524-9050, 1558-0016. DOI: 10.1109/TITS.2019.2910595

[25] M. Cordts és tsai., „The Cityscapes Dataset for Semantic Urban Scene Understanding”, *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA: IEEE, 2016. jún., 3213–3223. old., ISBN: 978-1-4673-8851-1. DOI: 10.1109/CVPR.2016.350

[26] F. Yu és tsai., „BDD100K: A Diverse Driving Dataset for Heterogeneous Multitask Learning”, *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Seattle, WA, USA: IEEE, 2020. jún., 2633–2642. old., ISBN: 978-1-7281-7168-5. DOI: 10.1109/CVPR42600.2020.00271

[27] D. Arya és tsai. „Crowdsensing-Based Road Damage Detection Challenge (CRDDC-2022)”, elérés dátuma 2026. febr. 26. cím: http://arxiv.org/abs/2211.11362 preprint.

[28] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiyama és H. Omata, „Road Damage Detection Using Deep Neural Networks with Images Captured Through a Smartphone”, *Computer-Aided Civil and Infrastructure Engineering*, 33. évf., 12. sz., 1127–1141. old., 2018. dec., ISSN: 1093-9687, 1467-8667. DOI: 10.1111/mice.12387

[29] D. Arya, H. Omata, H. Maeda és Y. Sekimoto, „ORDDC'2024: State of the Art Solutions for Optimized Road Damage Detection", *2024 IEEE International Conference on Big Data (BigData)*, Washington, DC, USA: IEEE, 2024. dec. 15., 8430–8438. old., ISBN: 979-8-3503-6248-0. DOI: 10.1109/BigData62323.2024.10825254

[30] F. Zhuang és tsai., „A Comprehensive Survey on Transfer Learning", *Proceedings of the IEEE*, 109. évf., 1. sz., 43–76. old., 2021. jan., ISSN: 0018-9219, 1558-2256. DOI: 10.1109/JPROC.2020.3004555

[31] J. Yosinski, J. Clune, Y. Bengio és H. Lipson, „How Transferable Are Features in Deep Neural Networks?", 1. verzió, 2014. DOI: 10.48550/ARXIV.1411.1792

[32] R. Szeliski, *Computer Vision: Algorithms and Applications* (Texts in Computer Science sor.). London: Springer, 2011, 1 old., ISBN: 978-1-84882-934-3 978-1-84882-935-0.

[33] M. Bertozz, A. Broggi és A. Fascioli, „Stereo inverse perspective mapping: Theory and applications", *Image and Vision Computing*, 16. évf., 8. sz., 585–590. old., 1998. jún., ISSN: 02628856. DOI: 10.1016/S0262-8856(97)00093-0

[34] M. Oliveira, V. Santos és A. D. Sappa, „Multimodal inverse perspective mapping", *Information Fusion*, 24. évf., 108–121. old., 2015. júl., ISSN: 15662535. DOI: 10.1016/j.inffus.2014.09.003

[35] J. Han, O. Heo, M. Park, S. Kee és M. Sunwoo, „Vehicle distance estimation using a mono-camera for FCW/AEB systems", *International Journal of Automotive Technology*, 17. évf., 3. sz., 483–491. old., 2016. jún. 1., ISSN: 1976-3832. DOI: 10.1007/s12239-016-0050-9

[36] T. R. Nezhad, E. Louback, R. Ahmed és A. Emadi, „Driving Simulators for Autonomous Vehicles: Comprehensive Review of Current Applications and Research Trends", *TechRxiv*, 2026. évf., 305. sz., DOI: 10.36227/techrxiv.177273746.67258655/v1

[37] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez és V. Koltun. „CARLA: An Open Urban Driving Simulator", elérés dátuma 2026. ápr. 30. cím: http://arxiv.org/abs/1711.03938 preprint.

[38] „Sensors Reference - CARLA Simulator", elérés dátuma 2026. máj. 1. cím: https://carla.readthedocs.io/en/latest/ref_sensors/

[39] „Python API - CARLA Simulator", elérés dátuma 2026. máj. 1. cím: https://carla.readthedocs.io/en/latest/python_api/

[40] „C++ Reference - CARLA Simulator", elérés dátuma 2026. máj. 1. cím: https://carla.readthedocs.io/en/latest/ref_cpp/

[41] T. H. Ji, Y. J. Kim, J. Y. Jung és C. Lee, „Integration and Validation of Trucksim Vehicle Model on Carla Simulator”, *International Journal of Automotive Technology*, 2026. febr. 20., ISSN: 1976-3832. DOI: 10.1007/s12239-026-00420-2

[42] A. Gambi, P. Maul, M. Mueller, L. Stamatogiannakis, T. Fischer és S. Panichella, „Soft-body Simulation and Procedural Generation for the Development and Testing of Cyber-physical Systems”,

[43] P. Maul, M. Mueller, F. Enkler, E. Pigova, T. Fischer és L. Stamatogiannakis, „BeamNG.tech Technical Paper”,

[44] K. KC, S. Worrall, J. S. Berrio és M. Balamurali, „CyberPotholes: Pothole Defect Detection Using Synthetic Depth-Maps”, *2024 IEEE 27th International Conference on Intelligent Transportation Systems (ITSC)*, 2024. szept., 2370–2377. old. DOI: 10.1109/ITSC58415.2024.10920234

[45] D. M. D. Carli, F. Bevilacqua, C. Tadeu Pozzer és M. C. d'Ornellas, „A Survey of Procedural Content Generation Techniques Suitable to Game Development”, *2011 Brazilian Symposium on Games and Digital Entertainment*, 2011, 26–35. old. DOI: 10.1109/SBGAMES.2011.15

[46] Z. Ma és S. Liu, „A review of 3D reconstruction techniques in civil engineering and their applications”, *Advanced Engineering Informatics*, 37. évf., 163–174. old., 2018. aug., ISSN: 14740346. DOI: 10.1016/j.aei.2018.05.005

[47] R. Schmidt, C. Grimm és B. Wyvill, „Interactive decal compositing with discrete exponential maps”, *ACM SIGGRAPH 2006 Papers on - SIGGRAPH '06*, Boston, Massachusetts: ACM Press, 2006, 605. old., ISBN: 978-1-59593-364-5. DOI: 10.1145/1179352.1141930

[48] M. K. Turkcan, Y. Li, C. Zang, J. Ghaderi, G. Zussman és Z. Kostic. „Boundless: Generating Photorealistic Synthetic Data for Object Detection in Urban Streetscapes”, elérés dátuma 2026. máj. 3. cím: http://arxiv.org/abs/2409.03022 preprint.

6 ÁBRAJEGYZÉK

2.1. Kátyún belüli árnyékok szemléltetése [1] 5

2.2. Zajos bináris kép [2] 7

2.3. Szűrt bináris kép [2] 7

2.4. A cikkben bemutatott módszer végeredménye [2] 8

2.5. Faster R-CNN felépítése [13] 13

2.6. 3D kátyú modell készítés Blenderben [44] 28

7 TÁBLAJEGYZÉK

2.1. YOLOv5 verziók összehasonlítása [19] 18

2.2. YOLOv8 és a kvantálás ereje [19] 18

2.3. YOLOv8 és v10 összehasonlítása [19] 19